ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hỏi đáp tự động sử dụng kỹ thuật sinh câu trả lời dựa trên tìm kiếm

NGUYỄN ĐĂNG HUY
Huy.ND204658@sis.hust.edu.vn

Ngành Khoa học máy tính

Giảng viên hướng dẫn: PGS.TS Nguyễn Thị Kim Anh _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 01/2025

LỜI CẢM ƠN

Lời đầu tiên, em xin được gửi lời cảm ơn sâu sắc và chân thành tới PGS. TS. Nguyễn Thị Kim Anh, người đã tận tình hướng dẫn và định hướng cho em để giúp em vượt qua những khó khăn trong quá trình thực hiện đồ án này. Em cũng xin được gửi lời cảm ơn tới các thầy cô giảng viên Đại học bách khoa Hà Nội nói chung và trường Công nghệ thông tin và Truyền Thông nói riêng đã truyền đạt cho em những kiến thức và hành trang quý báu giúp em trưởng thành hơn. Con cũng xin bày tỏ lòng biết ơn vô hạn tới bố mẹ và gia đình, nơi luôn là chỗ dựa tuyệt vời nhất với con. Đồng thời, em cũng xin gửi lời cảm ơn đến những người bạn đã gắn bó và đồng hành với em trong suốt 4 năm học thời sinh viên. Những trải nghiệm tuyệt vời khi là sinh viên Đại học bách khoa Hà Nội sẽ luôn khắc ghi trong tim em là những kỉ niệm mà em sẽ không bao giờ quên.

LỜI CAM KẾT

Họ và tên sinh viên: Nguyễn Đăng Huy
Điện thoại liên lạc: 0398457824
Email: huy.nd204658@sis.hust.edu.vn
Lớp: Khoa học máy tính 06
Hệ đào tạo: Kỹ Sư Chính Quy

Tôi – *Nguyễn Đăng Huy* – cam kết Đồ án Tốt nghiệp (ĐATN) là công trình nghiên cứu của bản thân tôi dưới sự hướng dẫn của *PGS.TS. Nguyễn Thị Kim Anh*. Các kết quả nêu trong ĐATN là trung thực, là thành quả của riêng tôi, không sao chép theo bất kỳ công trình nào khác. Tất cả những tham khảo trong ĐATN – bao gồm hình ảnh, bảng biểu, số liệu, và các câu từ trích dẫn – đều được ghi rõ ràng và đầy đủ nguồn gốc trong danh mục tài liệu tham khảo. Tôi xin hoàn toàn chịu trách nhiệm với dù chỉ một sao chép vi phạm quy chế của nhà trường.

Hà Nội, ngày tháng năm

Tác giả ĐATN

Nguyễn Đăng Huy

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sự phát triển mạnh mẽ của các mô hình ngôn ngữ lớn, tích hợp trong các hệ thống hỏi đáp tiên tiến như ChatGPT, Gemini,... đã mang lại nhiều lợi ích đáng kể, cho phép máy tính tương tác và hỗ trợ con người thực hiện nhiều tác vụ ngôn ngữ một cách hiệu quả hơn. Tuy nhiên, các hệ thống hỏi đáp sử dụng mô hình ngôn ngữ lớn phải đối mặt với nhiều thách thức như không có khả năng cập nhật kiến thức mới, câu trả lời thiếu độ tin cậy và nguy cơ tạo ra thông tin sai lệch. Kỹ thuật tìm kiếm thông tin để bổ sung kiến thức cho mô hình ngôn ngữ lớn, hay RAG (Retrieval-Augmented Generation), đã được phát triển như một giải pháp hiệu quả để khắc phục những hạn chế này. Dù vậy, mô hình vẫn còn gặp phải hạn chế trong việc xử lý các câu hỏi đòi hỏi suy luận nhiều bước hoặc các câu hỏi yêu cầu đọc hiểu văn bản dài, nơi mà chất lượng ngữ cảnh đầu vào đóng vai trò rất quan trọng. Đồ án này tập trung nghiên cứu phương pháp phân đoạn văn bản (chunking) nhằm nâng cao chất lượng phân đoạn, từ đó cải thiện độ chính xác của các phản hồi trong hệ thống hỏi đáp sử dụng kỹ thuật RAG. Trong quá trình thực hiện, đồ án đã thử nghiệm và đánh giá nhiều phương pháp phân đoạn, từ các kỹ thuật truyền thống đến hiện đại, đặc biệt chú trọng đến kỹ thuật sử dụng các mô hình ngôn ngữ lớn. Trên cơ sở đó, đồ án đề xuất giải pháp kết hợp ưu điểm của các phương pháp hiện có để phát triển một phương pháp phân đoạn tối ưu hơn.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1

1.1 Đặt vấn đề 1

1.2 Các giải pháp hiện tại và hạn chế 2

1.2.1 Phân đoạn dựa trên luật 3

1.2.2 Phân đoạn dựa trên ngữ nghĩa 4

1.3 Mục tiêu và định hướng giải pháp 4

1.4 Đóng góp của đồ án 5

1.5 Bố cục đồ án 5

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT 7

2.1 Mô hình ngôn ngữ lớn 7

2.1.1 Tổng quan về mô hình ngôn ngữ lớn 7

2.1.2 Một số dòng mô hình ngôn ngữ lớn: Gemini, Qwen, LLaMA 9

2.2 Kỹ thuật tạo tăng cường truy xuất 9

2.2.1 Tổng quan về kỹ thuật 10

2.2.2 Phân đoạn tài liệu 11

2.2.3 Truy xuất tài liệu dựa trên truy vấn người dùng 11

2.2.4 Sinh phản hồi dựa trên mô hình ngôn ngữ lớn 12

2.3 Các chiến lược tìm kiếm tài liệu 13

2.3.1 Tìm kiếm mức ngữ nghĩa: Vector Search 13

2.3.2 Tìm kiếm mức từ vựng: BM25 Search 14

2.4 Chiến lược xếp hạng lại tài liệu (re-rank) 14

2.5 Các nghiên cứu liên quan về phân đoạn văn bản 15

2.5.1 Phân đoạn với độ dài cố định 15

2.5.2 Phân đoạn theo ngữ nghĩa dựa trên điểm dừng với Semantic Chunker của thư viện LlamaIndex 15

2.5.3 RAPTOR 16

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT 18

3.1 Tổng quan giải pháp 18

3.2 Mô-đun phân đoạn tài liệu 18

3.2.1 Phân đoạn tài liệu bằng mô hình ngôn ngữ lớn 19

3.2.2 Kỹ thuật RAPTOR 21

3.3 Mô-đun truy xuất 23

3.3.1 Tìm kiếm tài liệu 24

3.3.2 Xếp hạng lại tài liệu được tìm kiếm 24

3.4 Mô-đun sinh câu trả lời 24

CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM 26

4.1 Các tham số đánh giá 26

4.1.1 BLEU 26

4.1.2 ROUGE-L 27

4.1.3 F1-Score 28

4.1.4 LLM-as-a-Judge (Đánh giá bằng mô hình ngôn ngữ lớn) 29

4.2 Các tiêu chí đánh giá 30

4.3 Thiết kế thử nghiệm 31

4.3.1 Tập dữ liệu đánh giá thử nghiệm 31

4.3.2 Cấu hình thử nghiệm 31

4.3.3 Các phương pháp tham chiếu 31

4.3.4 Các kịch bản thử nghiệm 32

4.4 Thử nghiệm và đánh giá hiệu năng của các kỹ thuật phân đoạn khác nhau với giải pháp đề xuất 33

4.5 Thử nghiệm và đánh giá hiệu năng của kỹ thuật RAPTOR kết hợp với các kỹ thuật phân đoạn khác nhau 34

4.6 Thử nghiệm và đánh giá của hiệu quả của các phương pháp hợp nhất phân đoạn. 35

4.7 Thử nghiệm và đánh giá mức độ ảnh hưởng của các chiến lược tìm kiếm và xếp hạng lại (rerank) khác nhau đối với giải pháp đề xuất 36

CHƯƠNG 5. KẾT LUẬN 38

5.1 Kết luận 38

5.2 Hướng phát triển trong tương lai 39

TÀI LIỆU THAM KHẢO 42

PHỤ LỤC 44

A. Hiệu quả tính toán của giải pháp đề xuất và các kỹ thuật tham chiếu .. 44

B. Một số ví dụ thực nghiệm 45

DANH MỤC HÌNH VẼ

Hình 1.1 Hiện tượng ảo giác trên mô hình Gemini 1.5 Flash, truy cập ngày 23/12/2024. 2
Hình 1.2 Minh họa vấn đề mất thông tin ngữ cảnh trong phân đoạn tài liệu. Một đoạn văn nói về chủ tịch Hồ Chí Minh được tách thành các đoạn. Phân đoạn có liên quan nhất đến truy vấn lại không chứa tên cụ thể của đối tượng được truy vấn. 3
Hình 1.3 Minh họa vấn đề hạn chế hiểu biết về toàn bộ tài liệu. Một đoạn văn liệt kê các đơn vị trực thuộc Bộ Giáo dục và Đào tạo được tách thành các đoạn. Các phân đoạn liên quan đến truy vấn chứa không đầy đủ thông tin khiến câu trả lời không chính xác. 5

Hình 2.1 Sự ra đời và phát triển của mô hình ngôn ngữ lớn qua các năm 8
Hình 2.2 Luồng hoạt động cơ bản của hệ thống RAG 10
Hình 2.3 Kiến trúc Bi-Encoder 13
Hình 2.4 Kiến trúc Cross-Encoder 14
Hình 2.5 Quy trình xây dựng cây: RAPTOR phân cụm đệ quy các đoạn văn bản dựa trên các vectơ nhúng của chúng và tạo ra các bản tóm tắt văn bản của các cụm đó, ... 16

Hình 3.1 Kiến trúc tổng quan hệ thống hỏi đáp tự động sử dụng kỹ thuật sinh câu trả lời dựa trên tìm kiếm 18
Hình 3.2 Giải pháp phân đoạn đề xuất 19
Hình 3.3 Phân đoạn tài liệu bằng mô hình ngôn ngữ lớn 20
Hình 3.4 Mẫu lời nhắc yêu cầu mô hình chọn một trong hai khả năng dựa trên cấu trúc logic và ngữ nghĩa nội dung 20
Hình 3.5 Tổng quan kỹ thuật RAPTOR 21
Hình 3.6 Mô-đun truy xuất tài liệu 23
Hình 3.7 Mô-đun sinh câu trả lời 24
Hình 3.8 Mẫu lời nhắc sinh câu trả lời 25

Hình B.1 Ví dụ khi thử nghiệm 45
Hình B.2 Ví dụ khi thử nghiệm với câu trả lời của giải pháp đề xuất . . . 46

DANH MỤC BẢNG BIỂU

Bảng 4.1 Thống kê bộ dữ liệu **HotPotQA** 31
Bảng 4.2 Thông tin cấu hình Kaggle 31
Bảng 4.3 Hiệu năng của giải pháp đề xuất và các kỹ thuật phân đoạn tham chiếu trên tập dữ liệu thử nghiệm 33
Bảng 4.4 Hiệu năng của các phương pháp khi kết hợp kỹ thuật RAPTOR 34
Bảng 4.5 Hiệu năng của các phương pháp hợp nhất phân đoạn 35
Bảng 4.6 Hiệu năng của các chiến lược tìm kiếm 36

Bảng A.1 Thời gian sinh câu trả lời và lượng ngữ cảnh đầu vào 44
Bảng A.2 Thời gian phân đoạn trung bình môt tài liệu trong tập dữ liệu thử nghiệm 44

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
BSC	Kỹ thuật phân đoạn theo ngữ nghĩa dựa trên điểm dừng (Breakpoint- based semantic Chunking)
FSC	Kỹ thuật phân đoạn với độ dài cố định (Fixed-size Chunking)
LBS	Kỹ thuật phân đoạn sử dụng mô hình ngôn ngữ lớn (LLM-based Chunking
LCS	Chuỗi con chung dài nhất (Longest Common Subsequence)
LLM	Mô hình ngôn ngữ lớn (Large Language Model
RAG	Kỹ thuật tạo tăng cường truy xuất (Retrieval-Augmented Generation)
ROUGE	Độ đo đánh giá sự tương đồng của hai văn bản (Recall-Oriented Understudy for Gisting Evaluation)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trí tuệ nhân tạo (AI, Artificial Intelligence) đang ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống, trở thành công cụ quan trọng giúp con người nâng cao năng suất làm việc và cải thiện chất lượng cuộc sống. Trong giai đoạn chuyển đổi số như hiện nay, Trí tuệ nhân tạo đóng vai trò then chốt, thúc đẩy sự đổi mới và tối ưu hóa các hoạt động kinh doanh, quản lý và vận hành. Một minh chứng rõ nét cho sự phát triển vượt bậc của nó chính là sự nổi lên mạnh mẽ của các mô hình ngôn ngữ lớn, tích hợp trong các hệ thống hỏi đáp tự động như ChatGPT, Gemini, ... với khả năng xử lý và tạo ngôn ngữ tự nhiên, cho phép máy tính tương tác và hỗ trợ con người một cách hiệu quả hơn bao giờ hết.

Với kích thước vô cùng lớn và được huấn luyện trên khối lượng dữ liệu khổng lồ, các mô hình ngôn ngữ lớn có khả năng học được cấu trúc ngôn ngữ phức tạp, nắm bắt ngữ nghĩa và tích lũy một lượng lớn tri thức thế giới (world knowledge) từ các nguồn dữ liệu mà chúng đã được "đọc" [1] . Điều này giúp chúng có thể thực hiện được đa dạng các tác vụ ngôn ngữ như tạo văn bản, tóm tắt thông tin, dịch thuật, trả lời câu hỏi... một cách tự nhiên và mạch lạc [2] . Sự phổ biến của các công cụ, như ChatGPT, cho thấy các mô hình ngôn ngữ lớn ngày càng được sử dụng rộng rãi để trả lời các câu hỏi thường ngày. Vì thế, độ chính xác của thông tin mà các mô hình này cung cấp ngày càng trở nên quan trọng. Tuy nhiên, các mô hình ngôn ngữ lớn thường gặp hiện tượng "ảo giác" (hallucination), nói cách khác, câu trả lời mà chúng tạo ra có vẻ trôi chảy và mạch lạc, nhưng lại chứa thông tin không chính xác nên rất khó để phát hiện được những thông tin sai lệch này [3]. Hình 1.1 minh họa một ví dụ về hiện tượng ảo giác của các mô hình ngôn ngữ lớn. Trong ví dụ này, mô hình dù đưa ra lời giải thích có vẻ hợp lý "tên bốn người là bốn mùa trong năm" nhưng lại không đúng (vì người thứ tư tên là Nam).

Hiện tượng ảo giác hình thành trong suốt quá trình tiếp thu tri thức của các mô hình ngôn ngữ lớn, thường được bắt nguồn từ ba vấn đề chính: dữ liệu, quá trình huấn luyện và suy luận [3]. Một số vấn đề về dữ liệu có thể kể đến bao gồm: i) thông tin sai lệch và thành kiến trong nguồn dữ liệu huấn luyện, ii) chất lượng kém, dữ liệu trùng lặp làm ảnh hưởng đến hiệu năng mô hình, và đặc biệt là iii) giới hạn về kiến thức khi tri thức của các mô hình ngôn ngữ lớn dừng lại tại thời điểm huấn luyện chúng (ví dụ, kiến thức của mô hình GPT-4o được cập nhật đến tháng 10 năm 2023 [4]), cũng như mô hình không tiếp cận được các nội dung bị hạn chế bởi luật bản quyền [5].

Hinh_ve/AogiacLLM-Vidu1.png

Hình 1.1: Hiện tượng ảo giác trên mô hình Gemini 1.5 Flash, truy cập ngày 23/12/2024.

Để giảm thiểu hiện tượng ảo giác của các mô hình ngôn ngữ lớn, kỹ thuật tìm kiếm thông tin để bổ sung kiến thức cho mô hình, hay RAG (Retrieval-Augmented Generation), được đề xuất nhằm tích hợp thông tin từ các nguồn dữ liệu bên ngoài, bổ sung hoặc thay thế cho kiến thức có sẵn của mô hình, từ đó cung cấp câu trả lời chính xác và đáng tin cậy [6]. Nhìn chung, kỹ thuật này cung cấp kịp thời kiến thức cho các mô hình ngôn ngữ lớn, cải thiện khả năng giải quyết các câu hỏi yêu cầu thông tin mang tính thời điểm hoặc có tính chuyên môn cao. Trong RAG, việc chia nhỏ tài liệu thành các đoạn (chunk) là yếu tố ảnh hưởng trực tiếp đến chất lượng truy xuất và trả lời. Việc phân đoạn tài liệu giúp mô hình tập trung vào những phần thông tin liên quan nhất đến truy vấn, từ đó cải thiện độ chính xác và tốc độ xử lý, nhưng việc lựa chọn kích thước và kỹ thuật phân đoạn tối ưu là vấn đề cần được quan tâm. Một số thách thức trong việc phân đoạn tài liệu có thể kể đến như: i) phân đoạn quá ngắn làm mất mát thông tin ngữ cảnh của các đoạn (một câu có ý nghĩa khác nhau phụ thuộc vào các câu trước đó), ii) phân đoạn quá dài khiến thông tin bị dư thừa và có thể giảm độ chính xác mô hình do có nhiều thông tin "nhiễu", và iii) khó khăn trong việc xác định quy tắc phân đoạn phù hợp, mỗi cách phân đoạn khác nhau có thể dẫn đến kết quả khác nhau và mức độ mất mát thông tin khác nhau.

Việc thử nghiệm và đánh giá nhiều kỹ thuật để xác định quy tắc phân đoạn tối ưu là bước quan trọng trong quá trình triển khai. Từ thực tiễn trên, đồ án này sẽ tập trung phân tích, thử nghiệm và đánh giá các kỹ thuật phân đoạn hiện tại, từ đó đề xuất giải pháp phù hợp nhằm cải thiện chất lượng phân đoạn.

1.2 Các giải pháp hiện tại và hạn chế

Các kỹ thuật phân đoạn hiện tại có thể được chia thành hai nhóm chính: phân đoạn dựa trên luật (rule-based chunking) và dựa trên ngữ nghĩa (semantic-based

chunking).

1.2.1 Phân đoạn dựa trên luật

Hai kỹ thuật dựa trên luật phổ biến và đơn giản nhất để phân đoạn tài liệu là *phân đoạn với kích thước cố định* (fixed-size chunking) và *phân đoạn dựa trên dấu phân cách* (recursive chunking). Trong khi kỹ thuật phân đoạn với kích thước cố định tiến hành chia tài liệu thành các đoạn có kích thước bằng nhau dựa trên một số lượng ký tự hoặc từ được xác định trước, phân đoạn dựa trên dấu phân cách lại thực hiện tách các đoạn dựa trên các dấu phân tách được xác định trước (như ký tự xuống dòng, dấu chấm câu,. . .).

Mặc dù các kỹ thuật này đơn giản và hiệu quả về mặt tính toán, nhưng nó có thể tách rời các câu có liên quan về ngữ cảnh, làm mất mát thông tin và giảm chất lượng truy xuất [7] (xem Hình 1.2). Để duy trì tính liên tục của ngữ cảnh, hướng tiếp cận phổ biến là sử dụng kỹ thuật cửa sổ trượt (sliding window chunking), cho phép chồng lấn (overlapping) các phân đoạn liền kề (hay hai phân đoạn liên tiếp có một phần nội dung trùng nhau). Phần chồng lấn này đóng vai trò như một "vùng đệm" ngữ cảnh, giúp các đoạn liền kề chia sẻ thông tin và duy trì sự liên kết với nhau.

Hinh_ve/Picture18.png

Hình 1.2: Minh họa vấn đề mất thông tin ngữ cảnh trong phân đoạn tài liệu. Một đoạn văn nói về chủ tịch Hồ Chí Minh được tách thành các đoạn. Phân đoạn có liên quan nhất đến truy vấn lại không chứa tên cụ thể của đối tượng được truy vấn.

Một vấn đề cần lưu tâm là xác định độ dài phần chồng lấn tối ưu. Phần chồng lấn quá ngắn có thể dẫn đến mất mát thông tin ngữ cảnh, ngược lại, phần chồng lấn quá lớn sẽ tạo ra các phân đoạn rất dài, làm tăng chi phí tính toán và giảm hiệu quả truy xuất. Bên cạnh đó, vì không sử dụng thông tin ngữ nghĩa, các kỹ thuật phân đoạn dựa trên luật thường không thể nhận biết được ranh giới tự nhiên giữa các ý hoặc chủ đề trong tài liệu. Điều này dẫn đến việc một số ý chính trong tài liệu có thể bị chia cắt giữa các phân đoạn, làm giảm khả năng nắm bắt đầy đủ thông tin

của từng đoạn.

1.2.2 Phân đoạn dựa trên ngữ nghĩa

Nhóm các kỹ thuật phân đoạn dựa trên ngữ nghĩa thực hiện chia tài liệu thành các đoạn dựa trên ý nghĩa và ngữ cảnh, đảm bảo mỗi đoạn là một đơn vị thông tin hoàn chỉnh, thể hiện một chủ đề hoặc một ý tưởng cụ thể. Kỹ thuật được sử dụng phổ biến là *phân đoạn ngữ nghĩa dựa trên điểm dừng* (breakpoint-based semantic chunking), được đề xuất bởi Greg Kamradt năm 2024 [8] . Thay vì chỉ dựa trên dấu phân cách, kỹ thuật này quyết định việc ngắt đoạn dựa trên mức độ tương đồng về mặt ngữ nghĩa của câu (hoặc nhóm câu) phía trước và câu (hoặc nhóm câu) phía sau vị trí dấu phân cách đang xét [9] . Mức độ tương đồng thấp (dưới một ngưỡng được cấu hình trước) cho thấy sự thay đổi về chủ đề, ngữ cảnh hoặc ý nghĩa giữa các câu (hoặc nhóm câu), từ đó xác định được ranh giới phân đoạn một cách chính xác hơn. Điều này cho phép tạo ra các phân đoạn mạch lạc và liên kết chặt chẽ về mặt ngữ nghĩa, tuy nhiên không nắm bắt được sự phụ thuộc của các phân đoạn có khoảng cách xa. Để khắc phục điều này, kỹ thuật *phân đoạn ngữ nghĩa dựa trên phân cụm* (clustering-based semantic chunking) sử dụng thuật toán phân cụm để nhóm các câu có mức độ ngữ nghĩa tương đồng với nhau [10] . Kỹ thuật này cho phép các câu không liên tiếp có thể nằm trong một phân đoạn, nhưng lại mất thông tin ngữ cảnh giữa các câu liên tiếp trong tài liệu.

Tuy nhiên, hệ thống sử dụng các kỹ thuật trên chỉ truy xuất được những đoạn ngắn, làm hạn chế sự hiểu biết toàn diện về ngữ cảnh của toàn bộ tài liệu. Điều này đặc biệt gây khó khăn khi xử lý các truy vấn đòi hỏi tích hợp thông tin từ nhiều phần khác nhau trong văn bản, chẳng hạn như hiểu toàn bộ tài liệu (minh họa ở Hình 1.3). Cần có các kỹ thuật tiên tiến hơn để khắc phục vấn đề này.

1.3 Mục tiêu và định hướng giải pháp

Đồ án hướng đến mục tiêu sau:

1. Tìm hiểu và nghiên cứu các phương pháp, kỹ thuật phân đoạn khác nhau, từ đó đưa ra đánh giá và các phương pháp kết hợp để tận dụng ưu điểm của các phương pháp cũng như hạn chế, nhược điểm phù hợp với tình huống đặt ra về tài nguyên sử dụng.

2. Đưa ra đề xuất cải tiến và hiệu quả có khả năng làm nâng cao hiệu quả trong mô đun truy xuất.

3. Cài đặt hệ thống và thử nghiệm với các kịch bản khác nhau. Nêu và phân tích được những kết quả thực nghiệm, so sánh đánh giá giữa các phương pháp và đưa ra các ưu điểm và nhược điểm của các kỹ thuật thử nghiệm.

Hinh_ve/òểãnh13.png

Hình 1.3: Minh họa vấn đề hạn chế hiểu biết về toàn bộ tài liệu. Một đoạn văn liệt kê các đơn vị trực thuộc Bộ Giáo dục và Đào tạo được tách thành các đoạn. Các phân đoạn liên quan đến truy vấn chứa không đầy đủ thông tin khiến câu trả lời không chính xác.

Trên cơ sở các phân tích và đánh giá ở phần 1.2 và để đạt được những mục tiêu đã nêu phía trên, đồ án này sẽ tập trung vào các khía cạnh:

1. Thực hiện kết hợp phương pháp phân đoạn cổ điển với phương pháp phân đoạn sử dụng mô hình ngôn ngữ lớn để cải thiện chất lượng phân đoạn.

2. Thiết kế luồng truy xuất hiệu quả bằng cách kết hợp các phương pháp truy xuất và sử dụng thêm mô-đun xếp hạng để cải thiện thêm chất lượng truy xuất.

1.4 Đóng góp của đồ án

Đồ án này có 2 đóng góp chính như sau:

1. Đồ án đề xuất giải pháp kết hợp các kỹ thuật phân đoạn khác nhau nhằm tăng hiệu suất của hệ thống truy xuất thông tin.
2. Thực hiện thử nghiệm kết hợp các kỹ thuật truy xuất nhằm cải thiện kết quả đầu ra.

1.5 Bố cục đồ án

Toàn bộ báo cáo đồ án tốt nghiệp được triển khai trong 5 chương. Các chương còn lại của báo cáo có nội dung như sau.

Chương 2 đề cập đến các nội dung lý thuyết nhằm phục vụ việc nghiên cứu, xây dựng thử nghiệm và đánh giá giải pháp đề xuất. Trong chương này, tôi sẽ trình bày tổng quan về mô hình ngôn ngữ lớn, các ứng dụng, hạn chế và một số dòng mô hình ngôn ngữ lớn phổ biến. Kỹ thuật RAG với các thành phần và các giải pháp hiện có cũng sẽ được phân tích chi tiết ở chương này.

Chương 3 trình bày chi tiết về giải pháp đề xuất. Trước hết, tôi mô tả tổng quan

về luồng xử lý, sau đó là đi sâu vào từng mô-đun. Trong mô-đun phân đoạn, tôi trình bày hai kỹ thuật phân đoạn tôi lấy làm ý tưởng đó là *phân đoạn sử dụng mô hình ngôn ngữ lớn* và *RAPTOR*. Sau đó, tôi đề xuất việc kết hợp hai kỹ thuật này để bổ trợ cho nhau. Trong mô-đun truy xuất, tôi trình bày việc kết hợp hai kỹ thuật đó là: i) tìm kiếm mức ngữ nghĩa và ii) tìm kiếm mức từ vựng nhằm cải thiện mức độ phù hợp của các tài liệu tìm kiếm được.

Chương 4 trình bày cụ thể về các kịch bản thử nghiệm, thông số cấu hình thử nghiệm, kết quả thực nghiệm và các đánh giá, nhận xét về các phương pháp thử nghiệm. Trong chương này, tôi sử dụng một số độ đo tự động thường được sử dụng cho hỏi đáp và đánh giá bằng mô hình ngôn ngữ lớn. Những nhận xét và đánh giá hiệu năng của phương pháp đề xuất so với các phương pháp tham chiếu cũng được trình bày tại chương này.

Chương 5 là chương cuối cùng. Trong chương này, tôi nêu ra kết luận về phương pháp đề xuất, những ưu điểm cũng như những hạn chế còn tồn tại cũng như đề ra các hướng phát triển trong tương lai.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

Trong chương này, tôi sẽ trình bày về các kiến thức nền tảng phục vụ cho quá trình thiết kế và xây dựng giải pháp đề xuất. Nội dung bao gồm tổng quan về mô hình ngôn ngữ lớn, tiếp đến là sự ra đời của *kỹ thuật RAG*. Tôi sẽ đi vào chi tiết từng thành phần trong một hệ thống hỏi đáp sử dụng *kỹ thuật RAG*. Cuối cùng, những phân tích và đánh giá của tôi về các công trình nghiên cứu có liên quan đến phân đoạn sẽ được nêu ở phần 2.5.

2.1 Mô hình ngôn ngữ lớn

2.1.1 Tổng quan về mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (Large Language Models - LLM) là các mô hình học sâu được huấn luyện trên một lượng dữ liệu khổng lồ nhằm "hiểu" và tạo ra ngôn ngữ tự nhiên. Các mô hình này thường dựa trên kiến trúc học sâu, đặc biệt là Transformer [11], để nắm bắt các mối quan hệ phức tạp giữa các từ, câu và đoạn văn bản. Transformer, được giới thiệu lần đầu bởi Vaswani và cộng sự vào năm 2017, đã trở thành nền tảng cho hầu hết các mô hình ngôn ngữ lớn hiện đại nhờ khả năng xử lý song song và hiệu quả trong việc hiểu ngữ cảnh dài hạn.

Các loại mô hình ngôn ngữ lớn hiện này thường được tiếp cận bởi người dùng cũng như những nhà phát triển theo hai hướng chính:

- **Mô hình mã nguồn mở:** Các mô hình như Qwen và LLaMA thuộc loại mã nguồn mở, cho phép người dùng tùy chỉnh và phát triển thêm theo nhu cầu cụ thể. Điều này mang lại tính linh hoạt cao, hỗ trợ nghiên cứu và ứng dụng đa dạng. Tuy nhiên, việc triển khai các mô hình này đòi hỏi kiến thức kỹ thuật chuyên sâu và tài nguyên tính toán đáng kể.
- **Mô hình mã nguồn đóng:** Các mô hình như GPT là các mô hình mã nguồn đóng, được phát triển và duy trì bởi các tổ chức lớn như OpenAI,... Những mô hình này dễ sử dụng và thường có chất lượng cao, tuy nhiên kèm theo những han chế khả năng tùy chỉnh và thường là các mô hình trả phí.

Cách thức hoạt động cơ bản: LLM hoạt động bằng cách dự đoán từ tiếp theo trong một chuỗi ngôn ngữ dựa trên ngữ cảnh trước đó. Quá trình huấn luyện của các mô hình này bao gồm hai giai đoạn chính: tiền huấn luyện (pre-training) và tinh chỉnh (fine-tuning). Trong giai đoạn tiền huấn luyện, mô hình học cách dự đoán từ tiếp theo dựa trên một lượng lớn dữ liệu đa dạng, từ đó nắm bắt cấu trúc ngôn ngữ và tích lũy tri thức thế giới. Giai đoạn tinh chỉnh giúp mô hình thích ứng với các nhiệm vụ cụ thể bằng cách huấn luyện thêm trên các tập dữ liệu đặc thù.

Ứng dụng của mô hình ngôn ngữ lớn: Mô hình ngôn ngữ lớn ngày càng chứng minh được khả năng vượt trội và được ứng dụng rộng rãi trong đời sống hiện nay. Một trong những ứng dụng quan trọng nhất của mô hình ngôn ngữ lớn là hệ thống hỏi đáp (QA), nơi chúng có thể cung cấp các câu trả lời chính xác và tự nhiên cho các câu hỏi của người dùng. Ngoài ra, mô hình này còn được sử dụng để tóm tắt văn bản, tạo ra các bản tóm tắt ngắn gọn và mạch lạc từ các tài liệu dài, giúp tiết kiệm thời gian và công sức xử lý thông tin. Trong lĩnh vực dịch thuật, chúng hỗ trợ chuyển đổi ngôn ngữ một cách chính xác và tự nhiên, góp phần làm cầu nối giữa các nền văn hóa. Không chỉ dừng lại ở đó, mô hình ngôn ngữ lớn còn được ứng dụng để sáng tác nội dung, tạo ra các bài viết, bài báo hoặc các nội dung sáng tạo khác, mở ra nhiều cơ hội mới trong các lĩnh vực truyền thông và sáng tạo nội dung.

Thách thức: Bên cạnh những lợi ích to lớn và tính ứng dụng ngày càng đa dạng, mô hình ngôn ngữ lớn vẫn tồn đọng những hạn chế nhất định có thể kể đến là khả năng cập nhật kiến thức mới sau khi quá trình huấn luyện đã hoàn tất. Do mô hình chỉ biết đến thông tin đến thời điểm huấn luyện, việc tích hợp tri thức mới đòi hỏi phải huấn luyện lại hoặc tinh chỉnh mô hình, điều này tiêu tốn nhiều thời gian và tài nguyên. Ngoài ra, việc đảm bảo chất lượng và tính chính xác của dữ liệu mới cũng là một thách thức không nhỏ.

Chuong/LLM-Evolutionary-Tree.png

Hình 2.1: Sự ra đời và phát triển của mô hình ngôn ngữ lớn qua các năm

Hình 2.1 minh hoạ sự phát triển nhanh chóng và bùng nổ của mô hình ngôn ngữ

lớn. Như ta có thể thấy từ năm 2023 đến nay, số lượng mô hình ngôn ngữ lớn ra đời bùng nổ và tăng rất nhanh. Điều này cho thấy sự quan tâm của toàn thế giới đang là rất lớn và dự đoán cho thấy đây sẽ là xu hướng của thế giới trong tương lai gần.

2.1.2 Một số dòng mô hình ngôn ngữ lớn: Gemini, Qwen, LLaMA

Gemini: Được phát triển bởi Google DeepMind, Gemini tập trung vào các ứng dụng thương mại với khả năng học liên tục và tối ưu hóa xử lý ngôn ngữ tự nhiên. Gemini nổi bật với khả năng tích hợp sâu vào các dịch vụ của Google, cung cấp các giải pháp thông minh cho doanh nghiệp và người dùng cá nhân. Một số phiên bản nổi bật bao gồm Gemini 1.5 và Gemini Pro, mỗi phiên bản đều cải thiện hiệu suất và tính năng so với phiên bản trước.

Qwen: Được phát triển bởi Alibaba, Qwen hướng tới các doanh nghiệp với khả năng hỗ trợ đa ngữ và quản lý thông tin hiệu quả. Qwen được thiết kế để đáp ứng nhu cầu xử lý ngôn ngữ tự nhiên trong môi trường kinh doanh, cung cấp các công cụ mạnh mẽ cho việc phân tích dữ liệu và tương tác khách hàng. Các phiên bản như Qwen-Enterprise và Qwen-MultiLingual đã được triển khai rộng rãi trong các ứng dụng thương mại điện tử và dịch vụ khách hàng.

LLaMA: Là một mô hình mã nguồn mở đến từ Meta, LLaMA hỗ trợ nghiên cứu và phát triển các ứng dụng AI mà không phụ thuộc vào các nền tảng mã nguồn đóng. Phiên bản mới như LLaMA3.2 đã cải thiện đáng kể hiệu suất, mở rộng khả năng ứng dụng trong nhiều lĩnh vực khác nhau, từ nghiên cứu học thuật đến các ứng dụng công nghiệp. LLaMA nổi bật với tính linh hoạt và khả năng tùy chỉnh cao, cho phép các nhà phát triển dễ dàng tích hợp vào các hệ thống hiện có.

Các mô hình này đại diện cho sự tiến bộ không ngừng trong lĩnh vực trí tuệ nhân tạo, mỗi dòng mô hình đều có những đặc điểm và ứng dụng riêng, đáp ứng nhu cầu đa dạng của người dùng và doanh nghiệp. Sự phát triển liên tục của LLM không chỉ cải thiện khả năng hiểu và tạo ngôn ngữ tự nhiên mà còn mở ra nhiều cơ hội mới trong việc ứng dụng AI vào các lĩnh vực khác nhau.

2.2 Kỹ thuật tạo tăng cường truy xuất

Kỹ thuật tạo tăng cường truy xuất (Retrieval Augmented Generation - RAG) là một phương pháp tiên tiến trong lĩnh vực trí tuệ nhân tạo, kết hợp giữa khả năng truy xuất thông tin từ các nguồn dữ liệu bên ngoài và khả năng tạo ra câu trả lời tự nhiên của các mô hình ngôn ngữ lớn (Large Language Models - LLM). Phương pháp này nhằm mục đích cải thiện độ chính xác và độ tin cậy của các hệ thống hỏi đáp bằng cách cung cấp thông tin bổ sung và cập nhật từ các tài liệu bên ngoài, giúp mô hình ngôn ngữ lớn tránh được hiện tượng "ảo giác" (hallucination) và cung cấp câu trả lời dựa trên dữ liệu thực tế.

2.2.1 Tổng quan về kỹ thuật

Kỹ thuật tạo tăng cường truy xuất (Retrieval Augmented Generation - RAG) được phát triển nhằm khắc phục một số hạn chế của các mô hình ngôn ngữ lớn (LLM) trong việc xử lý các câu hỏi đòi hỏi thông tin chi tiết, cập nhật hoặc có tính chuyên môn cao. Một trong những vấn đề chính của LLM là hiện tượng "ảo giác" (hallucination), nơi các mô hình có thể đưa ra các câu trả lời mạch lạc nhưng không chính xác, dựa trên thông tin sai lệch hoặc không tồn tại. Nguyên nhân của vấn đề này nằm ở việc LLM chỉ dựa trên tri thức đã được học trong quá trình huấn luyện, vốn bị giới hạn bởi thời điểm và nội dung dữ liệu huấn luyện.

RAG giải quyết vấn đề này bằng cách bổ sung khả năng truy xuất thông tin từ các nguồn dữ liệu bên ngoài, giúp mô hình có thể cập nhật thông tin và tăng độ tin cậy của câu trả lời. Thay vì chỉ dựa vào tri thức nội tại của mô hình, RAG tận dụng cả khả năng tìm kiếm thông tin trong cơ sở dữ liệu hoặc tài liệu bên ngoài, sau đó tích hợp thông tin này vào quá trình sinh câu trả lời. Điều này không chỉ cải thiện độ chính xác mà còn mở rộng phạm vi ứng dụng của LLM trong các lĩnh vực như hỏi đáp chuyên sâu, phân tích dữ liệu, và hỗ trợ ra quyết định.

Kiến trúc cơ bản của hệ thống RAG:

Hệ thống RAG bao gồm ba mô-đun chính hoạt động theo luồng tuần tự, như minh họa trong Hình 2.2:

Hinh_ve/test.png

Hình 2.2: Luồng hoạt động cơ bản của hệ thống RAG

Trong kiến trúc RAG, mỗi mô-đun đóng vai trò quan trọng trong việc đảm bảo rằng hệ thống có thể cung cấp các câu trả lời chính xác và đáng tin cậy. Mô-đun phân đoạn tài liệu không chỉ giúp tối ưu hóa hiệu quả truy xuất mà còn giảm tải cho các bước xử lý sau. Mô-đun truy xuất tài liệu đảm bảo rằng hệ thống chỉ xử lý những đoạn tài liệu có liên quan nhất đến truy vấn, giúp tăng tốc độ và độ chính xác của hệ thống. Cuối cùng, mô-đun sinh phản hồi kết hợp thông tin từ các đoạn

truy xuất với truy vấn của người dùng để tạo ra câu trả lời, đảm bảo rằng kết quả trả về vừa chính xác, vừa tự nhiên.

Nhờ kiến trúc này, RAG không chỉ khắc phục được hạn chế của các mô hình ngôn ngữ lớn mà còn tạo tiền đề cho nhiều ứng dụng thông minh, như trợ lý ảo, hệ thống hỏi đáp chuyên sâu, và hỗ trợ nghiên cứu.

2.2.2 Phân đoạn tài liệu

Phân đoạn tài liệu là một bước quan trọng trong quá trình xử lý thông tin, nhằm chia nhỏ tài liệu gốc thành các đoạn ngắn hơn, mỗi đoạn mang một ý nghĩa logic hoặc ngữ cảnh cụ thể. Mục tiêu chính của việc phân đoạn là tăng hiệu quả truy xuất bằng cách giảm kích thước dữ liệu cần xử lý, đảm bảo rằng hệ thống chỉ tập trung vào những phần thông tin liên quan nhất đến truy vấn. Việc này không chỉ giúp giảm tải tính toán mà còn duy trì được mối liên hệ ngữ nghĩa và ngữ cảnh cần thiết để hệ thống truy xuất được đúng nội dung.

Các kỹ thuật phân đoạn phổ biến hiện nay bao gồm phân đoạn kích thước cố định, phân đoạn theo ngữ nghĩa và phân đoạn dựa trên mô hình ngôn ngữ lớn. Phân đoạn kích thước cố định là một phương pháp đơn giản, chia tài liệu thành các đoạn có kích thước bằng nhau dựa trên số lượng câu hoặc token. Tuy nhiên, kỹ thuật này có thể tách rời các câu liên quan ngữ nghĩa, dẫn đến mất mát thông tin quan trọng. Trong khi đó, phân đoạn theo ngữ nghĩa tập trung vào việc giữ nguyên liên kết ngữ nghĩa giữa các câu, nhưng yêu cầu phân tích phức tạp và đòi hỏi tài nguyên tính toán lớn hơn. Kỹ thuật hiện đại hơn là sử dụng mô hình ngôn ngữ lớn để xác định điểm phân đoạn tối ưu dựa trên ngữ cảnh và cấu trúc ngôn ngữ, mang lại sự linh hoạt và chính xác cao.

Quá trình phân đoạn thường bắt đầu bằng việc tách tài liệu thành các câu riêng lẻ. Các câu này sau đó được nhóm lại thành các đoạn dựa trên các tiêu chí như độ dài tối đa hoặc ngữ cảnh ngữ nghĩa. Các đoạn văn bản cuối cùng sẽ được đánh chỉ mục và lưu trữ trong cơ sở dữ liệu, chuẩn bị cho quá trình truy xuất thông tin. Dù có nhiều ưu điểm như tăng hiệu quả xử lý và duy trì ngữ cảnh, các phương pháp phân đoạn vẫn đối mặt với thách thức trong việc cân bằng giữa tính toàn vẹn ngữ nghĩa và hiệu quả tính toán.

2.2.3 Truy xuất tài liệu dựa trên truy vấn người dùng

Trong kiến trúc hệ thống hỏi đáp dựa trên kỹ thuật RAG, quá trình truy xuất tài liệu dựa trên truy vấn người dùng đóng vai trò quyết định trong việc tìm kiếm và lựa chọn thông tin liên quan. Quá trình này bao gồm hai bước chính: truy xuất và xếp hạng lại. Mục tiêu là đảm bảo rằng chỉ những đoạn văn bản có liên quan nhất đến truy vấn của người dùng được chọn để cung cấp thông tin đầu vào cho mô hình

ngôn ngữ lớn sinh câu trả lời.

Bước đầu tiên là tìm kiếm, trong đó hệ thống sử dụng các thuật toán để xác định các đoạn văn bản liên quan trong cơ sở dữ liệu. Hai kỹ thuật phổ biến nhất là Vector Search và Key-word Search. Vector Search chuyển đổi văn bản thành các vec-tơ số, sau đó so sánh sự tương đồng giữa các vec-tơ này để tìm ra các đoạn văn bản phù hợp. Phương pháp này đặc biệt hiệu quả trong việc hiểu ngữ cảnh và mối quan hệ ngữ nghĩa giữa các từ và câu. Ngược lại, Key-word Search là một thuật toán tìm kiếm truyền thống dựa trên tần suất từ và độ hiếm của từ trong tài liệu, thường được sử dụng để xử lý nhanh các truy vấn cơ bản.

Sau bước tìm kiếm, các kết quả được xếp hạng lại dựa trên mức độ liên quan và chất lượng của chúng. Việc xếp hạng lại này sử dụng các mô hình học sâu hoặc các thuật toán tiên tiến hơn để đảm bảo rằng các đoạn văn bản quan trọng nhất được ưu tiên. Xếp hạng lại không chỉ cải thiện độ chính xác mà còn giúp hệ thống cung cấp câu trả lời phù hợp và tin cậy hơn.

2.2.4 Sinh phản hồi dựa trên mô hình ngôn ngữ lớn

Sau khi truy xuất được các đoạn văn bản liên quan, bước tiếp theo là sinh phản hồi dựa trên mô hình ngôn ngữ lớn. Đây là giai đoạn hệ thống kết hợp thông tin từ các đoạn văn bản được truy xuất với truy vấn của người dùng để tạo ra một câu trả lời chính xác và tự nhiên. Quá trình này phụ thuộc nhiều vào chất lượng của lời nhắc (prompt) được thiết kế để hướng dẫn mô hình ngôn ngữ trong việc tạo câu trả lời.

Lời nhắc là một đoạn văn bản được thiết kế để tích hợp thông tin từ tài liệu và truy vấn của người dùng. Chúng được sử dụng để cung cấp ngữ cảnh và định hướng cho mô hình ngôn ngữ trong việc xử lý truy vấn. Một lời nhắc hiệu quả cần rõ ràng, chính xác và đầy đủ thông tin để mô hình hiểu đúng yêu cầu và trả lời phù hợp. Việc tạo lời nhắc (Prompt Engineering) đòi hỏi kỹ năng và sự hiểu biết sâu sắc về cách mô hình xử lý ngôn ngữ tự nhiên, nhằm tối ưu hóa hiệu quả của câu trả lời.

Trong quá trình sinh phản hồi, hệ thống đầu tiên tạo ra lời nhắc bằng cách kết hợp truy vấn của người dùng với các đoạn văn bản được truy xuất. Lời nhắc này sau đó được gửi đến mô hình ngôn ngữ lớn để tạo câu trả lời. Cuối cùng, kết quả sẽ được kiểm tra và xử lý nếu cần thiết trước khi trả về cho người dùng. Quá trình này không chỉ đảm bảo rằng câu trả lời chính xác mà còn duy trì được tính tự nhiên và mạch lạc trong ngôn ngữ.

Tuy nhiên, sinh phản hồi dựa trên mô hình ngôn ngữ lớn cũng đối mặt với một số thách thức. Chẳng hạn, nếu lời nhắc không được thiết kế tốt, câu trả lời có thể

không chính xác hoặc không phù hợp với truy vấn. Ngoài ra, việc đảm bảo rằng mô hình hiểu và xử lý đúng ngữ cảnh của câu hỏi là một thách thức không nhỏ, đặc biệt trong các trường hợp đòi hỏi thông tin phức tạp hoặc đa tầng.

2.3 Các chiến lược tìm kiếm tài liệu

2.3.1 Tìm kiếm mức ngữ nghĩa: Vector Search

Hinh_ve/bi-encoder.png

Hình 2.3: Kiến trúc Bi-Encoder

Hình 2.3 minh họa cách Bi-Encoder được sử dụng trong VectorSearch để tạo các vector đại diện ngữ nghĩa cho truy vấn và tài liệu. Quá trình này cho phép đo lường độ tương đồng giữa các vector, hỗ trợ tìm kiếm thông tin hiệu quả dựa trên ý nghĩa thay vì chỉ dựa vào từ khóa.

Tổng quan phương pháp: VectorSearch là một kỹ thuật tìm kiếm dựa trên việc đo lường sự tương đồng ngữ nghĩa giữa truy vấn và tài liệu trong cơ sở dữ liệu. Phương pháp này sử dụng các mô hình nhúng (embedding models) như BERT hoặc Sentence Transformers để biểu diễn cả truy vấn và tài liệu dưới dạng các vectơ trong không gian nhiều chiều. Độ tương đồng giữa các vectơ này được đo lường, thường bằng cosine similarity, để xác định mức độ liên quan.

Ưu điểm và nhược điểm: VectorSearch có khả năng truy xuất thông tin dựa trên ý nghĩa ngữ cảnh, không bị giới hạn bởi sự xuất hiện trực tiếp của từ khóa, giúp khắc phục các hạn chế của các phương pháp tìm kiếm truyền thống. Tuy nhiên,

phương pháp này yêu cầu tài nguyên tính toán lớn và phụ thuộc nhiều vào chất lượng của các embedding, điều này có thể trở thành thách thức khi áp dụng cho cơ sở dữ liệu lớn hoặc dữ liệu không đồng nhất. Bên cạnh đó, khoảng cách trong không gian vectơ đôi khi không phản ánh chính xác ngữ nghĩa trong các trường hợp phức tạp.

2.3.2 Tìm kiếm mức từ vựng: BM25 Search

Tổng quan phương pháp: BM25 (Best Matching 25) là một thuật toán tìm kiếm dựa trên tần suất từ (term frequency - TF) và độ nghịch đảo của tần suất tài liệu (inverse document frequency - IDF). Phương pháp này đánh giá mức độ liên quan giữa tài liệu và truy vấn dựa trên sự xuất hiện của các từ khóa từ truy vấn trong tài liệu, với các trọng số điều chỉnh để tối ưu hóa hiệu quả tìm kiếm.

Ưu điểm và nhược điểm: BM25 dễ triển khai, yêu cầu tài nguyên tính toán thấp và phù hợp với các hệ thống có cơ sở dữ liệu lớn. Phương pháp này hoạt động tốt khi từ khóa xuất hiện rõ ràng trong tài liệu, mang lại hiệu quả cao cho các truy vấn cơ bản. Tuy nhiên, BM25 không thể hiểu được ngữ cảnh ngữ nghĩa sâu và có thể bỏ sót các tài liệu liên quan nếu từ khóa không xuất hiện trực tiếp. Điều này làm cho phương pháp kém hiệu quả hơn khi xử lý các ngôn ngữ phức tạp hoặc truy vấn có ý nghĩa đa tầng.

2.4 Chiến lược xếp hạng lại tài liệu (re-rank)

Hinh_ve/cross_encoder.png

Hình 2.4: Kiến trúc Cross-Encoder

Hình 2.4 minh họa cơ chế Cross-Encoder được sử dụng trong mô-đun re-ranking để đánh giá mức độ liên quan giữa truy vấn và tài liệu. Mô hình xử lý cặp truy vấn-

tài liệu đồng thời, tận dụng thông tin ngữ nghĩa và cú pháp để xếp hạng lại kết quả tìm kiếm một cách chính xác hơn.

Tổng quan phương pháp: Xếp hạng lại tài liệu (re-ranking) là một bước nâng cao, được thực hiện sau khi các tài liệu đã được truy xuất sơ bộ. Quá trình này đánh giá lại mức độ liên quan của các tài liệu dựa trên các tiêu chí nâng cao hoặc sử dụng các mô hình học sâu để tối ưu hóa thứ tự kết quả tìm kiếm. Các mô hình re-ranking thường được thiết kế để hiểu sâu hơn về ngữ nghĩa và cú pháp, từ đó cung cấp kết quả phù hợp hơn với truy vấn của người dùng.

Ưu điểm và nhược điểm: Chiến lược re-ranking giúp cải thiện đáng kể chất lượng kết quả tìm kiếm, đặc biệt trong các trường hợp truy vấn phức tạp hoặc yêu cầu độ chính xác cao. Phương pháp này tận dụng các mô hình học sâu để xử lý các yếu tố ngữ nghĩa và cú pháp, từ đó mang lại các kết quả có mức độ liên quan cao. Tuy nhiên, re-ranking yêu cầu nhiều tài nguyên tính toán và có thể làm tăng thời gian xử lý truy vấn. Hơn nữa, việc tối ưu hóa mô hình để đạt hiệu quả trong các tình huống cụ thể đòi hỏi thời gian và kiến thức chuyên sâu, gây khó khăn trong việc triển khai trên diện rộng.

2.5 Các nghiên cứu liên quan về phân đoạn văn bản

Phần này trình bày các nghiên cứu nổi bật liên quan đến phân đoạn văn bản và các phương pháp tiếp cận trong bối cảnh xử lý ngôn ngữ tự nhiên. Mỗi kỹ thuậ được phân tích tổng quan và đánh giá ưu, nhược điểm từ đó cung cấp ý tưởng nền tảng cho phương pháp đề xuất của tôi

2.5.1 Phân đoạn với độ dài cố định

Tổng quan phương pháp: Phương pháp này chia văn bản thành các đoạn (chunks) có độ dài cố định, thường được xác định bằng số từ hoặc ký tự. Mỗi đoạn có kích thước bằng nhau, bất kể nội dung hay ý nghĩa.

Đánh giá: Kỹ thuật này đơn giản, dễ triển khai và có tốc độ xử lý nhanh, phù hợp với dữ liệu lớn. Tuy nhiên, nó thường cắt ngang câu hoặc ý nghĩa, dẫn đến mất mạch nội dung và không phù hợp với các văn bản có cấu trúc ngữ nghĩa phức tạp. Để cải thiện vấn đề này, một số phương pháp được sử dụng sẽ là phân chia theo dấu câu hoặc thêm thành phần chồng chéo (overlap).

2.5.2 Phân đoạn theo ngữ nghĩa dựa trên điểm dừng với Semantic Chunker của thư viện LlamaIndex

Tổng quan phương pháp: Semantic Chunker trong LlamaIndex là một công cụ tiên tiến để phân đoạn văn bản dựa trên ngữ nghĩa. Thay vì chia văn bản thành các đoạn cố định, công cụ này sử dụng các mô hình nhúng ngữ nghĩa để xác định điểm

ngắt giữa các câu, đảm bảo mỗi đoạn chứa các câu liên quan về mặt ý nghĩa. Điều này giúp bảo toàn mạch nội dung và ý nghĩa cốt lõi của văn bản, đặc biệt hữu ích trong các ứng dụng xử lý ngôn ngữ tự nhiên. [8]

Đánh giá: Semantic Chunker giúp duy trì tính toàn vẹn ngữ nghĩa và cải thiện hiệu suất trong các bài toán như truy xuất thông tin và hỏi đáp. Tuy nhiên, phương pháp này gặp phải một số nhược điểm. Đầu tiên, nó thiếu tính linh hoạt khi phải xử lý các loại văn bản không đồng nhất. Thứ hai, Semantic Chunker có xu hướng tạo ra các đoạn (chunks) có kích thước dài không đồng đều, dẫn đến khó khăn trong việc kiểm soát và xử lý tiếp theo. Ngoài ra, công cụ này đòi hỏi tài nguyên tính toán lớn và phụ thuộc nhiều vào chất lượng mô hình nhúng ngữ nghĩa, làm tăng độ phức tạp trong triển khai.

2.5.3 RAPTOR

Hinh_ve/base_RAPTOR_RAG.png

Hình 2.5: Quy trình xây dựng cây: RAPTOR phân cụm đệ quy các đoạn văn bản dựa trên các vectơ nhúng của chúng và tạo ra các bản tóm tắt văn bản của các cụm đó,

Hình 2.5 mô tả kỹ thuật RAPTOR sử dụng để phân đoạn văn bản. Tổng quan về kỹ thuật này được tôi trình bày dưới đây:

Tổng quan phương pháp: RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) là một phương pháp hiện đại được đề xuất bởi **Parth Sarthi**. Phương pháp bao gồm kỹ thuật phân đoạn xây dựng cấu trúc cây phân cấp. Cụ thể, các đoạn văn bản ban đầu được chia nhỏ, nhúng vào không gian vector, sau đó nhóm lại thành các cụm dựa trên ngữ nghĩa và được tóm tắt thành các nút cha. Quá trình này lặp lại nhiều lần, tạo ra một cây tổ chức thông tin với nhiều mức độ trừu tượng khác nhau, từ chi tiết đến tổng quan. [12]

Đánh giá: RAPTOR đã chứng minh hiệu quả vượt trội trong các bài toán yêu cầu xử lý thông tin phức tạp và suy luận đa bước. Nhờ cấu trúc cây phân cấp, phương pháp này có khả năng tích hợp thông tin từ nhiều mức độ ngữ cảnh, giúp mô hình hiểu sâu sắc hơn về nội dung và cung cấp câu trả lời chính xác, toàn diện. Điều này đặc biệt hữu ích trong các bài toán như hỏi đáp trên văn bản dài hoặc

phân tích ngữ nghĩa sâu. Tuy nhiên, RAPTOR cũng tồn tại một số hạn chế. Việc xây dựng và tóm tắt đệ quy đòi hỏi tài nguyên tính toán lớn, dẫn đến chi phí cao hơn so với các phương pháp truy xuất truyền thống. Hơn nữa, cấu trúc phức tạp của RAPTOR yêu cầu người triển khai có kiến thức chuyên sâu về xử lý ngôn ngữ tự nhiên và hệ thống.

Kết chương

Trong chương này, báo cáo đã trình bày tổng quan về các thành phần quan trọng trong hệ thống hỏi đáp dựa trên kỹ thuật tạo tăng cường truy xuất (RAG). Đầu tiên, mô hình ngôn ngữ lớn (LLM) được giới thiệu như một nền tảng chính, với khả năng xử lý và sinh ngôn ngữ tự nhiên, nhưng cũng tồn tại các hạn chế như hiện tượng "ảo giác" và giới hạn tri thức tại thời điểm huấn luyện. Tiếp theo, luồng hoạt động cơ bản của RAG được phân tích, bao gồm ba mô-đun chính: phân đoạn văn bản, truy xuất tài liệu và sinh phản hồi.

Ngoài ra, các phương pháp phân đoạn văn bản cũng được nghiên cứu kỹ lưỡng, từ phân đoạn kích thước cố định đến phân đoạn theo ngữ nghĩa và sử dụng mô hình ngôn ngữ lớn. Dù mỗi phương pháp đều có những ưu điểm nhất định, như đơn giản và hiệu quả (phân đoạn cố định) hoặc khả năng duy trì ngữ nghĩa (phân đoạn ngữ nghĩa), chúng vẫn tồn tại các hạn chế. Ví dụ, phân đoạn kích thước cố định có thể làm mất ngữ cảnh quan trọng, trong khi phân đoạn ngữ nghĩa yêu cầu tính toán phức tạp và tài nguyên lớn.

Bên cạnh đó, các chiến lược tìm kiếm tài liệu cũng được trình bày, bao gồm tìm kiếm mức ngữ nghĩa bằng VectorSearch và tìm kiếm mức từ vựng bằng BM25 Search, cùng với chiến lược xếp hạng lại tài liệu (re-rank) để nâng cao chất lượng kết quả truy xuất. Mỗi phương pháp đều có ưu nhược điểm riêng, thể hiện rõ ràng qua phân tích trong chương.

Nhìn chung, các hạn chế như mất ngữ cảnh trong phân đoạn, sự phức tạp trong truy xuất ngữ nghĩa, và chi phí tính toán cao sẽ được giải quyết trong các chương tiếp theo thông qua việc đề xuất và cải tiến các phương pháp phù hợp. Những cải tiến này nhằm tối ưu hóa hiệu quả hệ thống, nâng cao độ chính xác và tính thực tiễn trong các ứng dụng hỏi đáp dựa trên RAG.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

Trong chương này, phần đầu sẽ mô tả tổng quan giải pháp đề xuất ứng dụng kỹ thuật tạo tăng cường truy xuất cho tác vụ hỏi đáp, và chi tiết từng thành phần trong hệ thống sẽ được trình bày trong các phần còn lại.

3.1 Tổng quan giải pháp

Hình 3.1 minh họa kiến trúc của một hệ thống hỏi đáp sử dụng kỹ thuật sinh câu trả lời dựa trên tìm kiếm. Hệ thống được xây dựng bao gồm ba mô-đun chính: mô-đun phân đoạn tài liệu, mô-đun truy xuất và mô-đun sinh câu trả lời.

Hinh_ve/Rag_pipeline.png

Hình 3.1: Kiến trúc tổng quan hệ thống hỏi đáp tự động sử dụng kỹ thuật sinh câu trả lời dựa trên tìm kiếm

Trước hết, tập ngữ liệu (corpus), đóng vai trò là nguồn tri thức bổ sung, được chia thành các phân đoạn nhằm phù hợp với giới hạn ngữ cảnh của mô hình ngôn ngữ [13] . Về chiến lược phân đoạn đề xuất, tôi sẽ trình bày chi tiết ở phần 3.2. Các phân đoạn này sau đó được mã hóa thành các biểu diễn vec-tơ số tương ứng bằng cách sử dụng mô hình nhúng và được lưu trữ trong một cơ sở dữ liệu (gọi là cơ sở dữ liệu vec-tơ) để quá trình truy xuất thông tin diễn ra nhanh chóng và hiệu quả. Với mỗi truy vấn người dùng, mô-đun truy xuất thực hiện một số quy trình để đo lường mức độ liên quan giữa truy vấn và các tài liệu trong cơ sở dữ liệu vec-tơ, từ đó đưa ra tập các tài liệu ứng viên có liên quan nhất đến truy vấn. Sau đó, các tài liệu ứng viên cùng truy vấn được tổng hợp thành một lời nhắc mạch lạc và gửi tới mô hình ngôn ngữ lớn để tạo phản hồi cho truy vấn tương ứng.

3.2 Mô-đun phân đoạn tài liệu

Phân đoạn tài liệu là một bước quan trọng trong kỹ thuật tạo tăng cường truy xuất, ảnh hưởng đến hiệu quả và độ chính xác trong quá trình truy xuất văn bản. Quá trình này nhằm chia nhỏ tài liệu gốc thành các đoạn ngắn hơn, mỗi đoạn mang

một ý nghĩa logic [13] . Tuy nhiên, hiệu quả của các chiến lược phân đoạn vẫn là một thách thức trong việc tối ưu hóa chất lượng truy xuất và hiệu quả tính toán [7].

Hinh_ve/Chunking Modul.png

Hình 3.2: Giải pháp phân đoạn đề xuất

Hình 3.2 mô tả tổng quan giải pháp phân đoạn tài liệu đề xuất. Đây là sự kết hợp của hai kỹ thuật: phân đoạn tài liệu dựa trên mô hình ngôn ngữ lớn (LLM-based chunking) và *kỹ thuật RAPTOR* [12] Đầu tiên, văn bản đầu vào sẽ được tách thành các câu, điều này giữ sự liền mạch ngữ nghĩa của câu văn. Tiếp đó, tập hợp các câu được tách sẽ được phân đoạn bằng mô hình ngôn ngữ lớn và sau đó áp dụng kỹ thuật RAPTOR trên tập phân đoạn thu được.

3.2.1 Phân đoạn tài liệu bằng mô hình ngôn ngữ lớn

Mặc dù *phân đoạn ngữ nghĩa dựa trên điểm dừng* tạo ra các phân đoạn mạch lạc và liên kết về mặt ngữ nghĩa. Tuy nhiên, kỹ thuật này còn hạn chế về tính linh hoạt. Cụ thể, nó phụ thuộc vào việc xác định trước một số lượng câu nhất định và một ngưỡng độ tương đồng cố định. Việc điều chỉnh hai siêu tham số đòi hỏi nhiều thử nghiệm tốn thời gian và công sức. Hơn nữa, việc chỉ xem xét một số lượng câu nhất định tại mỗi thời điểm làm hạn chế việc nắm bắt ngữ cảnh của đoạn văn bản đang xét. Ngoài ra, các phân đoạn được tạo ra có kích thước không đồng đều điều này dẫn đến khi truy xuất và đưa qua mô hình sinh có thể gây quá tải cho hệ thống với vài trường hợp.

Để khắc phục những hạn chế trên, đồ án đề xuất tiếp cận sử dụng mô hình ngôn ngữ lớn (Large Language Model - LLM) để cải thiện các hạn chế của phân đoạn ngữ nghĩa dựa trên điểm dừng. Bằng việc sử dụng lời nhắc, phương pháp này vừa linh hoạt hơn về số lượng câu xem xét tại mỗi thời điểm vừa nắm bắt toàn bộ ngữ

cảnh tốt hơn.

Hinh_ve/LLM_chunk.png

Hình 3.3: Phân đoạn tài liệu bằng mô hình ngôn ngữ lớn

Hình 3.3 mô tả quá trình phân đoạn tài liệu sử dụng mô hình ngôn ngữ lớn.

Luồng xử lý cụ thể của quá trình này như sau:

1. Tài liệu được tách thành các câu, đặt là các đoạn khởi tạo.
2. Xét đoạn thứ nhất và đoạn thứ hai. Hai đoạn này được tổng hợp thành một lời nhắc, sau đó đưa vào mô hình ngôn ngữ lớn để xem xét khả năng phân tách. Đặt trong giới hạn về hệ thống triển khai, tôi đề xuất lựa chọn mô hình **Qwen-1.5B-Instruct** [14], mô hình vừa đảm bảo nhỏ để có thể triển khai trên hệ thống với tài nguyên hạn chế vừa nổi bật với khả năng suy luận.

   Hình 3.4 mô tả mẫu lời nhắc được sử dụng cho quá trình này. Mô hình ngôn ngữ lớn sẽ lựa chọn một trong hai khả năng: : 1) giữ nguyên hai đoạn đang xét hoặc 2) hợp nhất chúng thành một đoạn.

This is a text chunking task. You are a text analysis expert. Please choose one of the following two options based on the logical structure and semantic content of the provided paragraph: 1. Split "{paragraph 1+ paragraph 2}" into "{paragraph 1}" and "{paragraph 2}" two parts; 2. Keep "{paragraph 1+ paragraph 2}" unsplit in its original form; Please answer 1 or 2.

Hình 3.4: Mẫu lời nhắc yêu cầu mô hình chọn một trong hai khả năng dựa trên cấu trúc logic và ngữ nghĩa nội dung

3. Nếu đoạn 1 và 2 được tách, thực hiện xem xét với đoạn 2 và 3. Ngược lại, nếu

đoạn 1 và 2 được hợp nhất, thực hiện xem xét đoạn hợp nhất này với đoạn 3. Quá trình này được lặp lại cho đến khi xử lý đến đoạn cuối cùng, từ đó thu được các đoạn văn bản được phân tách rõ ràng về mặt ngữ nghĩa.

3.2.2 Kỹ thuật RAPTOR

Mục tiêu của RAPTOR là giải quyết vấn đề không nắm bắt được sự phụ thuộc của các phân đoạn có khoảng cách xa trong các phương pháp phân đoạn truyền thống. Bằng cách bổ sung đoạn văn bản tóm tắt của từng cụm văn bản đã được nhóm với nhau theo độ tương đồng ngữ nghĩa, phương pháp này vừa giữ lại được các thông tin cốt lõi (thông qua những đoạn văn bản được phân đoạn theo cách thông thường), vừa bổ sung thêm ngữ cảnh tổng thể (thông qua những đoạn văn bản tóm tắt).

Hinh_ve/Raptor_general.png

Hình 3.5: Tổng quan kỹ thuật RAPTOR

Hình 3.5 mô tả tổng quan kỹ thuật RAPTOR, bao gồm tách câu, nhóm cụm và tóm tắt.

Phân đoạn. Văn bản đầu vào sẽ được tách theo một giới hạn số lượng token nhất định. Nếu một câu vượt quá giới hạn này, toàn bộ câu được chuyển sang đoạn tiếp theo để duy trì tính mạch lạc.

Nhóm cụm. Nhóm cụm là một phần đóng vai trò quan trọng trong phương pháp phân đoạn RAPTOR. GMM giả sử rằng các điểm dữ liệu được tạo ra từ sự kết hợp của các phân phối chuẩn Gauss. Xác suất một đoạn văn bản $\mathbf{x}$ thuộc về Gaussian thứ k được định nghĩa như sau:

$$P(\mathbf{x} \mid k) = \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k), \tag{3.1}$$

trong đó $\mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)$ là phân phối Gaussian nhiều chiều với các tham số:

- Trung bình μ_k
- Ma trận hiệp phương sai Σ_k

Xác suất tổng thể được tính bằng tổng trọng số của K thành phần Gaussian:

$$P(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k), \tag{3.2}$$

trong đó π_k là trọng số của thành phần Gaussian thứ k, thỏa mãn $\sum_{k=1}^{K} \pi_k = 1$.

Dữ liệu vector nhúng có số chiều lớn có thể gây cản trở hiệu suất phân cụm do khoảng cách trong không gian này không phản ánh chính xác sự tương đồng. RAPTOR sử dụng phương pháp UMAP (Uniform Manifold Approximation and Projection) để giảm chiều dữ liệu.

Số lượng cụm được tối ưu với tiêu chuẩn thông tin Bayes (BIC, Bayesian Information Criterion), được tính như sau:

$$\text{BIC} = \ln(N)k - 2\ln(\hat{L}), \tag{3.3}$$

trong đó:

- N: Số lượng đoạn văn bản.
- k: Số lượng tham số trong mô hình.
- $\hat{L}$: Giá trị hợp lý tối đa.

Giá trị BIC thấp nhất cho biết số cụm tối ưu, cân bằng giữa độ phức tạp của mô hình và mức độ phù hợp với dữ liệu.

Với số lượng cụm tối ưu do BIC xác định, thuật toán Expectation-Maximization sau đó được sử dụng để ước tính các tham số GMM, là các giá trị trung bình, hiệp phương sai và trọng số hỗn hợp.

Tóm tắt. Sau khi các đoạn văn bản được nhóm thành các cụm, các cụm này sau đó được đưa qua một mô hình ngôn ngữ lớn để tóm tắt với tiêu chí giữ lại được nhiều thông tin, việc này giúp chuyển đổi các khối văn bản lớn thành các bản tóm tắt ngắn gọn, mạch lạc. Cụ thể trong đồ án này tôi sử dụng mô hình **Gemini-1.5-flash** [15], mô hình với hiệu suất nhanh và hỗ trợ cửa sổ ngữ cảnh lên đến 1 triệu token. Ngoài ra, việc có thể truy cập mô hình này thông qua API giúp tiết kiệm tài nguyên, phù hợp với các hệ thống có giới hạn về nguyên xử lý.

Trên thực tế, sử dụng phương pháp bằng mô hình ngôn ngữ lớn thường thu được các đoạn văn bản có độ dài khá nhỏ do sự ổn định và chất lượng của mô hình. Mà mục tiêu của việc phân đoạn tài liệu là tạo ra các đoạn văn bản đủ lớn để có tính liền mạch nhưng cũng đủ nhỏ để tránh làm mô hình quá tải và chứa quá nhiều thông tin dư thừa [16]. Vì vậy, tôi thực hiện việc ghép các đoạn văn bản sau khi được phân đoạn bởi mô hình ngôn ngữ lớn. Các đoạn văn bản thuộc cùng một văn bản gốc sẽ được xử lý qua một vòng lặp để kiểm tra: nếu việc thêm đoạn văn bản kế tiếp không vượt quá số lượng token được thiết lập theo yêu cầu, thì hai đoạn sẽ được ghép lại thành một đoạn; ngược lại, hệ thống sẽ kiểm tra tương tự với đoạn tiếp theo. Quá trình này được lặp lại cho đến khi xử lý hết các đoạn văn bản cuối cùng của văn bản gốc. Kết quả thu được là các đoạn văn bản đã được ghép lại cùng với các đoạn tóm tắt, tạo thành tập hợp các đoạn văn bản cuối cùng.

3.3 Mô-đun truy xuất

Hinh_ve/Rerank.png

Hình 3.6: Mô-đun truy xuất tài liệu

Hình 3.6 mô tả kỹ thuật truy xuất tài liệu được sử dụng trong đồ án này. Quá trình truy xuất là quá trình tìm kiếm những đoạn văn bản từ kho văn bản được xử lý trong phần 3.2 sao cho những đoạn văn bản này là phù hợp và cung cấp thông tin để trả lời câu hỏi đầu vào. Mô-đun truy xuất được sử dụng trong đồ án bao gồm hai mô-đun nhỏ hơn lần lượt là: i) mô-đun tìm kiếm tài liệu (Retrieval Modul) và ii) mô-đun xếp hạng lại (Rerank Modul). Từ câu truy vấn, mô-đun tìm kiếm tài liệu sẽ thực hiện tìm kiếm và trả ra top k đoạn văn bản phù hợp với câu truy vấn dựa trên độ tương đồng. Tiếp đó, top k đoạn văn bản này được đưa qua mô-đun xếp hạng lại để sắp xếp lại top k đoạn văn bản này. Việc sử dụng hai mô-đun này bổ trợ cho nhau giúp đạt hiệu quả về cả hiệu suất và độ chính xác. Lý do là vì mặc dù độ chính xác khi sử dụng mô-đun xếp hạng lại cao do kiến trúc Cross-Encoder của mô-đun xếp hạng lại khi thực hiện suy luận đưa từng cặp (câu hỏi, văn bản) vào để tính toán độ tương đồng. Tuy nhiên, điều này dẫn đến thời gian tính toán tăng lên rất nhiều khi số lượng văn bản cần xếp hạng lớn. Ngược lại, các mô-đun tìm kiếm tài liệu mã hóa câu hỏi và văn bản độc lập, các vector nhúng của các đoạn văn bản có thể được tính toán trước và lưu trữ. Khi truy vấn mới đến, chỉ cần mã hóa truy vấn và tính toán độ tương đồng với các vector đã lưu. Sau khi qua mô-đun tìm kiếm tài liệu, danh sách các văn bản phù hợp được trả về đã được thu hẹp lại rất nhiều so

với kho đoạn văn bản gốc. Lúc này, mô-đun xếp hạng lại sẽ được sử dụng để xếp hạng lại các văn bản này, mang lại độ chính xác cao hơn.

3.3.1 Tìm kiếm tài liệu

Mô-đun tìm kiếm tài liệu được tôi sử dụng trong đồ án này là sự kết hợp của hai kỹ thuật tìm kiếm: i) tìm kiếm theo ngữ nghĩa và ii) tìm kiếm theo từ khóa với trọng số là 35 đoạn văn bản tìm kiếm theo ngữ nghĩa và 5 đoạn văn bản tìm kiếm theo từ khóa. Sự kết hợp này nhằm tận dụng ưu điểm của cả hai kỹ thuật tìm kiếm để nâng cao hiệu quả của quá trình tìm kiếm tài liệu. Tìm kiếm theo ngữ nghĩa giúp hiểu được ý nghĩa thực sự của các từ khóa trong ngữ cảnh , từ đó cung cấp kết quả chính xác hơn đối với các truy vấn phức tạp hoặc mơ hồ. Trong khi đó, tìm kiếm theo từ khóa lại giúp xác định nhanh chóng các tài liệu có chứa những từ ngữ , hữu ích trong việc tìm kiếm tài liệu có liên quan trực tiếp đến chủ đề hoặc chủ thể trong câu truy vấn. Cụ thể, với kỹ thuật tìm kiếm theo ngữ nghĩa tôi sử dụng mô hình nhúng *All-mpnet-base-v2* và thư viện *BM25s* [17] cho tất cả các kịch bản thử nghiệm.

3.3.2 Xếp hạng lại tài liệu được tìm kiếm

Mục đích của thành phần này là xếp hạng lại những kết quả được trả về từ thành phầm tìm kiếm tài liệu giúp tăng độ chính xác hơn.

Trong đồ án này, tất cả các thử nghiệm tôi đều sử dụng mô hình *Bge-rerank-v2-m3* cho thành phần xếp hạng lại. Sử dụng cùng một mô hình xếp hạng lại vừa làm cải thiện chất lượng của mô-đun tìm kiếm và đánh giá được chính xác hơn những ảnh hưởng mà các phương pháp phân đoạn văn bản tác động lên chất lượng câu trả lời.

3.4 Mô-đun sinh câu trả lời

Luồng hoạt động của mô-đun sinh câu trả lời được tôi trình bày qua hình 3.7 Ban đầu, câu truy vấn và các đoạn đoạn văn bản thích hợp được tích hợp với một

Hinh_ve/Generation.png

Hình 3.7: Mô-đun sinh câu trả lời

yêu cầu để tạo ra lời nhắc hoàn chỉnh cung cấp ngữ cảnh và nhiệm vụ cần thiết.

Sau khi lời nhắc này được gửi tới mô hình *Llama 3.2* tiếp đến mô hình này sẽ phân tích ngữ cảnh và thông tin trong lời nhắc, sử dụng khả năng suy luận của mình để sinh ra câu trả lời chính xác và tự nhiên.

Các thành phần chính trong mô-đun sinh câu trả lời

- **Prompt** là một thành phần quan trọng trong việc tương tác với các mô hình ngôn ngữ lớn. Prompt đóng vai trò ngữ cảnh được cung cấp cho mô hình để hướng dẫn mô hình tạo ra câu trả lời. Trong mô-đun này, prompt tạo ra từ câu hỏi và đoạn văn bản liên quan.

Answer the question based on the given passages. Only give me the answer and do not output any other words. The following are given passages. **Context** Answer the question based on the given passages. Only give me the answer and do not output any other words. Question: **Question** Answer:

Hình 3.8: Mẫu lời nhắc sinh câu trả lời

Hình 3.8 mô tả mẫu lời nhắc được sử dụng cho quá trình này.

- **Llama 3.2** là mô hình ngôn ngữ lớn được tôi lựa chọn trong mô-đun này bởi hiệu suất cao trong việc xử lý ngôn ngữ tự nhiên và là mô hình mã nguồn mở dễ dàng triển khai trên hệ thống cá nhân. Phiên bản LLama tôi sử dụng là *Llama3.2 - 3B Instruct*, với cấu trúc 3B tham số, mô hình cân bằng tốt giữa độ chính xác và khả năng xử lý nhanh, giúp giảm thiểu thời gian phản hồi mà vẫn duy trì được chất lượng cao.

CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM

Trong chương này, tôi sẽ trình bày các kết quả thực nghiệm để so sánh và đưa ra cái nhìn tổng thể về các phương pháp mà tôi đã đề cập và đề xuất. Trong phần đầu tiên tôi sẽ trình bày về việc thiết lập các tham số đánh giá (mục 4.1) và các tiêu chí đánh giá cho các thực nghiệm (mục 4.2). Trong phần tiếp theo tôi sẽ trình bày về việc lựa chọn các baseline để so sánh (mục 4.3.3) cũng như là đưa ra các phương án thử nghiệm (mục 4.3.4) và các phần còn lại tôi sẽ trình bày chi tiết về từng thử nghiệm.

4.1 Các tham số đánh giá

Hệ thống hỏi đáp có sử dụng mô hình ngôn ngữ lớn thường được đánh giá theo hai hướng: những độ đo tự động và mô hình đánh giá. Sau đây, tôi xin trình bày một số độ đo tự động và cách đánh giá bằng mô hình mà tôi đã sử dụng để đánh giá chất lượng đầu ra của hệ thống:

4.1.1 BLEU

BLEU (Bilingual Evaluation Understudy) [18] là một trong những độ đo tự động phổ biến nhất được sử dụng để đánh giá chất lượng của các hệ thống hỏi đáp. Được giới thiệu lần đầu bởi Papineni và cộng sự vào năm 2002, BLEU đã trở thành một tiêu chuẩn trong việc đo lường sự giống nhau giữa văn bản được tạo ra bởi máy và văn bản tham chiếu do con người viết.

BLEU hoạt động dựa trên việc so sánh các n-gram (chuỗi liên tiếp gồm n từ) trong câu đầu ra của hệ thống với các n-gram trong câu tham chiếu. Nguyên tắc chính bao gồm:

- **Độ chính xác của n-gram (n-gram Precision):** Được xác định bằng tỷ lệ các n-gram trong đầu ra mà xuất hiện trong câu tham chiếu. BLEU thường sử dụng từ 1-gram đến 4-gram để đảm bảo đánh giá cả ngữ pháp lẫn ý nghĩa.
- **Hình phạt độ ngắn (Brevity Penalty):** Được áp dụng để tránh việc hệ thống tạo ra các câu quá ngắn so với câu tham chiếu. Nếu độ dài của câu đầu ra ngắn hơn một ngưỡng nhất định so với câu tham chiếu, hình phạt này sẽ làm giảm điểm BLEU.

Công thức tính BLEU

Công thức tính BLEU bao gồm hai thành phần chính: độ chính xác n-gram và hình phạt độ ngắn. Cụ thể:

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^{N} w_n \cdot \log p_n\right) \tag{4.1}$$

Trong đó:

- BP là hình phạt độ ngắn.
- p_n là độ chính xác của n-gram.
- w_n là trọng số cho mỗi n-gram (thường bằng nhau).

Hình phạt độ ngắn được tính bằng:

$$\text{BP} = \begin{cases} 1 & \text{nếu } c > r \\ \exp\left(1 - \frac{r}{c}\right) & \text{nếu } c \leq r \end{cases}$$

Trong đó:

- c là độ dài của câu đầu ra.
- r là độ dài của câu tham chiếu gần nhất.

4.1.2 ROUGE-L

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence) [19] là một trong những độ đo đánh giá tự động phổ biến được sử dụng để đánh giá chất lượng của các hệ thống tóm tắt và hỏi đáp. ROUGE-L tập trung vào việc đánh giá sự giống nhau giữa câu trả lời của hệ thống và câu trả lời tham chiếu dựa trên chuỗi con chung dài nhất (Longest Common Subsequence).

Nguyên tắc cơ bản của ROUGE-L

ROUGE-L hoạt động dựa trên việc tìm kiếm chuỗi con chung dài nhất giữa hai văn bản: văn bản đầu ra của hệ thống (hypothesis) và văn bản tham chiếu (reference). Nguyên tắc chính bao gồm:

- **Chuỗi con chung dài nhất (LCS):** ROUGE-L sử dụng LCS để đo lường mức độ trùng khớp về cấu trúc và thứ tự từ giữa hai câu trả lời. LCS là chuỗi dài nhất mà các từ trong nó xuất hiện theo cùng thứ tự trong cả hai câu.
- **Precision (Độ chính xác):** Đo lường tỷ lệ các từ trong chuỗi con chung dài nhất (Longest Common Subsequence - LCS) so với tổng số từ trong câu trả lời của hệ thống (Hypothesis).

$$\text{Precision} = \frac{LCS(\text{Reference}, \text{Hypothesis})}{\text{Length of Hypothesis}}$$

- **Recall (Độ đầy đủ):** Đo lường tỷ lệ các từ trong LCS so với tổng số từ trong câu trả lời tham chiếu (Reference).

$$\text{Recall} = \frac{LCS(\text{Reference}, \text{Hypothesis})}{\text{Length of Reference}}$$

Công thức tính ROUGE-L

ROUGE-L F1 là hàm số hợp nhất giữa Precision và Recall, được tính bằng công thức sau:

$$\text{ROUGE-L F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trong đó:

- Precision là độ chính xác của chuỗi con chung dài nhất.
- Recall là độ đầy đủ của chuỗi con chung dài nhất.

Công thức này thể hiện sự cân bằng giữa Precision và Recall, giúp đưa ra một chỉ số tổng hợp phản ánh cả hai khía cạnh của việc đánh giá.

4.1.3 F1-Score

Đây là độ đo được đề cập trong [20] dùng để đánh giá tập dữ liệu tôi sử dụng. F1-Score là trung bình điều hòa giữa **Precision** (Độ Chính Xác) và **Recall** (Độ Đầy Đủ), cung cấp một cái nhìn cân bằng về khả năng của hệ thống trong việc trả lời chính xác và đầy đủ các câu hỏi được đặt ra.

Định nghĩa Precision và Recall

Trước khi hiểu về F1-Score, chúng ta cần nắm rõ hai thành phần chính là Precision và Recall được sử dụng ở đây:

- **Precision (Độ Chính Xác):** Đo lường tỷ lệ các từ chính xác mà hệ thống QA cung cấp đúng so với tổng số từ mà hệ thống đã đưa ra. Precision phản ánh mức độ chính xác của các câu trả lời mà hệ thống tạo ra.

$$\text{Precision} = \frac{\text{Số từ chung chính xác}}{\text{Tổng số từ của câu trả lời dự đoán}}$$

- **Recall (Độ Đầy Đủ):** Đo lường tỷ lệ các từ chính xác mà hệ thống QA có thể tìm thấy so với tổng số từ chính xác có thể có. Recall phản ánh khả năng của hệ thống trong việc không bỏ sót các thông tin quan trọng.

$$\text{Recall} = \frac{\text{Số từ chung chính xác}}{\text{Tổng số từ của câu trả lời tham chiếu}}$$

Công thức tính F1-Score

F1-Score là trung bình điều hòa giữa Precision và Recall, được tính bằng công thức sau:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1-Score cung cấp một chỉ số tổng hợp phản ánh cả hai khía cạnh của Precision và Recall, giúp đánh giá cân bằng giữa độ chính xác và độ bao phủ của hệ thống QA.

4.1.4 LLM-as-a-Judge (Đánh giá bằng mô hình ngôn ngữ lớn)

Large Language Models as a Judge [21] là một trong những phương pháp đánh giá mới và tiên tiến được sử dụng trong việc đánh giá chất lượng của các hệ thống hỏi đáp (Question Answering - QA). Thay vì dựa vào các độ đo truyền thống như Precision, Recall, F1-Score hay ROUGE, phương pháp này sử dụng các mô hình ngôn ngữ lớn để đánh giá và so sánh câu trả lời của hệ thống với câu trả lời tham chiếu.

Tổng quan về LLM-As-a-Judge

LLM As a Judge là phương pháp sử dụng khả năng hiểu và phân tích ngữ cảnh của các mô hình ngôn ngữ lớn để đánh giá chất lượng câu trả lời của hệ thống QA. Các mô hình này được huấn luyện trên lượng lớn dữ liệu ngôn ngữ tự nhiên, cho phép chúng hiểu được ngữ cảnh, ý nghĩa, và độ tự nhiên của câu trả lời.

Tại sao sử dụng LLM-As-a-Judge

- **Hiểu ngữ cảnh tốt hơn:** Các mô hình ngôn ngữ lớn có khả năng hiểu sâu sắc ngữ cảnh và ý nghĩa của câu trả lời, vượt xa khả năng của các độ đo truyền thống chỉ dựa trên từ vựng.
- **Đánh giá ngữ nghĩa:** Khả năng đánh giá ngữ nghĩa giúp LLM As a Judge không chỉ kiểm tra sự trùng khớp từ ngữ mà còn xem xét mức độ phù hợp và tự nhiên của câu trả lời.
- **Linh hoạt và đa dạng:** Phương pháp này có thể áp dụng cho nhiều loại câu trả lời khác nhau, từ ngắn gọn đến chi tiết, mà không bị giới hạn bởi các n-gram như BLEU hay ROUGE.

Cách thức hoạt động của LLM-As-a-Judge

LLM As a Judge hoạt động bằng cách so sánh câu trả lời của hệ thống QA với câu trả lời tham chiếu thông qua các bước sau:

1. **Chuẩn bị dữ liệu:** Chuẩn bị câu trả lời của hệ thống và câu trả lời tham chiếu đi kèm câu hỏi.
2. **Phân tích ngữ cảnh:** Sử dụng LLM để phân tích ngữ cảnh và ý nghĩa của cả hai câu trả lời.
3. **Đánh giá và so sánh:** LLM đánh giá mức độ tương đồng về ngữ nghĩa, cấu trúc câu, và độ tự nhiên của câu trả lời.
4. **Gán điểm đánh giá:** LLM gán một điểm số phản ánh chất lượng câu trả lời dựa trên các tiêu chí đã định. Cụ thể, nếu hai câu trả lời mô tả cùng một nội dung sẽ được 1 điểm, ngược lại câu hỏi đó nhận về 0 điểm.

Kết quả cuối cùng của phương pháp đánh giá này được tính như sau:

$$\text{LLM-Score} = \frac{\text{Số câu mô hình đánh giá đúng}}{\text{Tổng số câu thử nghiệm}}$$

4.2 Các tiêu chí đánh giá

Trong phần này, tôi đề cập đến các tiêu chí đánh giá cho tác vụ hỏi đáp để từ đó đưa ra những góc nhìn về việc sử dụng các kỹ thuật phân đoạn khác nhau.

1. Chất lượng câu trả lời. Đây là tiêu chí ưu tiên của hệ thống. Các kịch bản được thử nghiệm đều hướng đến mục tiêu đầu tiên là cải thiện chất lượng câu trả lời.
2. Thời gian sinh câu trả lời, độ dài ngữ cảnh và tài nguyên sử dụng. Đây là các tiêu chí tỷ lệ với nhau cụ thể lượng ngữ cảnh đầu vào càng cao thì thời gian sinh câu trả lời càng lâu và lượng tài nguyên cần để xử lý càng lớn. Đây cũng là một tiêu chí cần xem xét khi đặt trong tình huống có giới hạn về mặt tài nguyên xử lý.
3. Thời gian phân đoạn văn bản. Như đã đề cập phần phía trên, các kỹ thuật phân đoạn sẽ cần lượng thời gian thực hiện là khác nhau. Đây là tiêu chí ít cần quan tâm hơn do được thực hiện ở pha "offline" tuy nhiên đây cũng là một chỉ số để có sự đánh giá thêm về các kỹ thuật phân đoạn.

4.3 Thiết kế thử nghiệm

4.3.1 Tập dữ liệu đánh giá thử nghiệm

Hệ thống được đánh giá trên tập dữ liệu **HotPotQA** cho tác vụ hỏi đáp suy luận đa bước được đề cập trong [22]. Bộ dữ liệu bao gồm 200 đoạn văn bản được lấy từ các đoạn văn Wikipedia hoàn chỉnh và ghép lại để tạo thành văn bản dài bao gồm cả các đoạn văn hỗ trợ hoặc những đoạn gây mất tập trung. Để kiểm thử khả năng đọc hiểu ngữ cảnh dài và trả lời câu hỏi đòi hỏi suy luận đa bước, bộ dữ liệu cung cấp 200 cặp câu hỏi và câu trả lời. **Bảng 4.1** dưới đây phân tích về độ dài của bộ dữ liệu.

Thống kê	**Avg. Len**
Độ dài trung bình văn bản	9151
Độ dài trung bình câu hỏi	19.525
Độ dài trung bình câu trả lời	3.775

Bảng 4.1: Thống kê bộ dữ liệu **HotPotQA**

Để điều chỉnh cho hệ thống RAG, tôi sử dụng cách chia các văn bản có ngữ cảnh dài trên thành các đoạn văn tương ứng riêng lẻ theo [23]. Vì mỗi đoạn văn là một đoạn văn Wikipedia hoàn chỉnh và mạch lạc về mặt ngữ nghĩa. Sau khi loại bỏ trùng lặp, các đoạn văn từ tất cả các câu hỏi tạo thành kho dữ liệu C gồm 1715 đoạn văn bản được sử dụng để tiến hành thử nghiệm tiếp theo.

4.3.2 Cấu hình thử nghiệm

Dưới đây là mô tả về cấu hình thử nghiệm nền tảng Kaggle dùng chung cho các kịch bản thử nghiệm:

System	**Information**
GPU	1 x Tesla P100, 3584 CUDA cores, 16GB GDDR6 VRAM
CPU	1 x single core hyper threaded Xeon Processors @2.2Ghz (1 core 2 threads)
RAM	~29 GB Available
Disk	~73.1 GB Available

Bảng 4.2: Thông tin cấu hình Kaggle

4.3.3 Các phương pháp tham chiếu

Giải pháp phân đoạn đề xuất được so sánh với các kỹ thuật phân đoạn khác như sau:

1. **Phân đoạn văn bản dựa trên độ dài cố định (FSC, Fixed-size chunking)**: Phương pháp mà tôi sử dụng ở đây là phân đoạn dựa trên số lượng token cố

định. Để duy trì sự liền mạch ngữ nghĩa trong các đoạn văn bản, quá trình phân đoạn ưu tiên chia theo câu. Cụ thể, khi thêm một câu vào một đoạn hiện tại, hệ thống sẽ kiểm tra tổng số token của đoạn đó có vượt quá độ dài đã định trước hay không. Nếu tổng số token sau khi thêm câu mới vượt quá giới hạn, câu đó sẽ được đẩy sang đoạn tiếp theo. Đồng thời, để đảm bảo sự liên kết giữa các đoạn, hai câu cuối cùng của đoạn trước sẽ được lặp lại trong đoạn tiếp theo (overlap = 2).

Cách tiếp cận này đảm bảo rằng mỗi đoạn văn bản không chỉ tuân thủ giới hạn về số lượng token mà còn giữ được cấu trúc ngữ nghĩa tự nhiên của văn bản bằng cách phân chia tại các ranh giới câu. Việc sử dụng overlap giúp duy trì ngữ cảnh giữa các đoạn, từ đó nâng cao chất lượng phân đoạn và hỗ trợ tốt hơn cho các bước xử lý tiếp theo bởi các mô hình ngôn ngữ lớn.

2. **Phân đoạn ngữ nghĩa dựa trên điểm dừng (BSC, breakpoint- based semantic chunking) [8]**: Trong đồ án này, tôi áp dụng kỹ thuật phân đoạn ngữ nghĩa dựa trên điểm dừng thông qua thư viện LlamaIndex, một công cụ mạnh mẽ hỗ trợ xử lý và phân đoạn văn bản dựa trên ngữ nghĩa. Thay vì phân đoạn văn bản với kích thước khối cố định, bộ chia ngữ nghĩa thích ứng chọn điểm ngắt giữa các câu bằng cách sử dụng tính tương đồng nhúng. Điều này đảm bảo rằng một "khối" chứa các câu có liên quan đến nhau về mặt ngữ nghĩa.

3. **RAPTOR [12]**: Luồng xử lý về kỹ thuật này được tiến hành bằng cách như sau: Đầu tiên, văn bản được phân đoạn theo kỹ thuật Fixed-sized Chunking, nhằm chia nhỏ tài liệu thành các đoạn văn bản có kích thước cố định. Việc phân đoạn này giúp chuẩn hóa dữ liệu và tạo điều kiện thuận lợi cho các bước xử lý tiếp theo. Sau đó, các đoạn văn bản được đưa vào quá trình nhóm cụm (clustering) dựa trên đặc trưng ngữ nghĩa, sử dụng các thuật toán phân cụm phù hợp để nhóm các đoạn văn bản có nội dung tương tự lại với nhau. Tiếp theo, sử mô hình ngôn ngữ lớn *Gemini -1.5- flash* để tóm tắt những cụm này. Cuối cùng, bổ sung những đoạn văn bản tóm tắt cùng với những đoạn văn bản ban đầu tạo thành tập đoạn văn bản được sử dụng cho các mục đích tìm kiếm và lưu trữ.

4.3.4 Các kịch bản thử nghiệm

Để so sánh và đánh giá ảnh hưởng của các kỹ thuật phân đoạn văn bản lên chất lượng của quá trình truy xuất và sinh câu trả lời, tôi xây dựng các kịch bản thử nghiệm dưới đây:

1. Thử nghiệm và đánh giá hiệu năng của các kỹ thuật phân đoạn khác nhau với giải pháp đề xuất.

2. Thử nghiệm và đánh giá hiệu năng của kỹ thuật RAPTOR kết hợp với các phương pháp phân đoạn khác nhau.
3. Thử nghiệm và đánh giá mức độ ảnh hưởng của các chiến lược hợp nhất phân đoạn trong giải pháp phân đoạn sử dụng mô hình ngôn ngữ lớn.
4. Thử nghiệm và đánh giá mức độ ảnh hưởng của các chiến lược tìm kiếm và xếp hạng lại (rerank) khác nhau đối với giải pháp đề xuất.

Để làm ngắn gọn tên các kỹ thuật phân đoạn trong thử nghiệm, tôi xin định nghĩa và chú thích các tên kỹ thuật như sau:

- Fixed-Size Chunking: FSC
- Breakpoint-based semantic chunking: BSC
- LLM-based Chunking: LBS
- Giải pháp đề xuất là sự kết hợp của kỹ thuật LBS, kỹ thuật RAPTOR và hợp nhất phân đoạn ở ngưỡng 400 tokens : PPDX

4.4 Thử nghiệm và đánh giá hiệu năng của các kỹ thuật phân đoạn khác nhau với giải pháp đề xuất

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	Rouge-L	F1	LLM-Score
PPDX	**46.64**	**28.32**	**19.93**	**14.41**	**55.41**	**54.07**	**63.5**
RAPTOR	42.82	24.02	16.79	12.32	51.35	50.5	60
LBC	41.82	24.69	16.56	12.42	49.48	49.11	56.5
BSC	43,89	25.9	18.83	13.57	51.99	51.53	62
FSC	42.48	25	17,22	12.24	50.10	49.31	57

Bảng 4.3: Hiệu năng của giải pháp đề xuất và các kỹ thuật phân đoạn tham chiếu trên tập dữ liệu thử nghiệm

Kết quả đánh giá các kỹ thuật được thể hiện trong **Bảng 4.3**. Tất cả các kỹ thuật đều được thử nghiệm chung một mô hình nhúng, một mô hình xếp hạng và một mô hình sinh. Số liệu của từng kỹ thuật phân đoạn cũng là kết quả cao nhất trong các trường hợp thử nghiệm với top k khác nhau.

Kết quả của giải pháp đề xuất đạt giá trị cao nhất ở tất cả các độ đo cho thấy hiệu năng của giải pháp đề xuất cao hơn. Với độ đo Rouge-L và độ đo F1 giải pháp đề xuất lần lượt tăng 6.5% và 4.88% so với giải pháp sử dụng kỹ thuật phân đoạn BSC trong khi đó số lượng ngữ cảnh đầu vào một câu hỏi cần cung cấp cho mô hình sinh của phương pháp đề xuất trung bình là 2243 tokens so với 2530 tokens

của giải pháp sử dụng kỹ thuật BSC, điều này thể hiện chất lượng đoạn văn bản sau khi phân đoạn của giải pháp đề xuất tốt hơn. So sánh với các giải pháp tham chiếu khác, giải pháp đề xuất cũng có hiệu năng vượt trội hơn hẳn. Điều này chỉ ra rằng giải pháp đề xuất nâng cao độ chính xác và tính trôi chảy của câu trả lời được tạo ra đối với câu trả lời tham chiếu.

Kỹ thuật LLM-base chunking có hiệu quả thấp hơn so với các phương pháp khác điều này có thể lý giải như sau: Thực nghiệm cho thấy việc các đoạn văn bản được phân đoạn bằng mô hình ngôn ngữ lớn có số lượng token khá nhỏ. Mặc dù các đoạn văn bản được phân đoạn bởi mô hình ngôn ngữ lớn thường tách biệt tốt về nội dung nhưng việc thiếu ngữ cảnh mở rộng có thể làm giảm khả năng mô hình truy xuất các đoạn liên quan bổ sung hoặc tạo kết nối giữa các đoạn hỗ trợ trả lời câu hỏi. Hơn nữa, chất lượng và độ ổn định của mô hình cũng làm ảnh hưởng đến chất lượng của việc phân đoạn Chính vì vậy, việc bổ sung thêm những đoạn văn bản tóm tắt và hợp nhất các phân đoạn là cần thiết để cải thiện chất lượng cho các phương pháp này.

4.5 Thử nghiệm và đánh giá hiệu năng của kỹ thuật RAPTOR kết hợp với các kỹ thuật phân đoạn khác nhau

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	Rouge-L	F1	LLM-Score
FSC	42.48	25	17,22	12.24	50.10	49.31	57
RAPTOR (FSC)	**42.82**	24.02	**16.79**	**12.32**	**51.35**	**50.5**	**60**
BSC	43.89	25.9	18.83	13.57	51.99	51.53	62
RAPTOR (BSC)	**44.01**	**26.33**	18.26	13.07	**53.89**	**53.12**	**62.5**
LBC	41.82	24.69	16.56	12.42	49.48	49.11	56.5
RAPTOR (LBC)	**42.60**	**24.90**	**16.57**	**12.80**	**50.41**	**49.52**	**59**

Bảng 4.4: Hiệu năng của các phương pháp khi kết hợp kỹ thuật RAPTOR

Bảng 4.4 mô tả kết quả khi so sánh hiệu năng của kỹ thuật Raptor kết hợp với các phương pháp phân đoạn khác nhau. Tất cả các kỹ thuật đều được thử nghiệm chung một mô hình nhúng, một mô hình xếp hạng và mô hình sinh. Số liệu của từng phương pháp cũng là kết quả cao nhất trong các trường hợp thử nghiệm với top k khác nhau.

Ta có thể thấy việc bổ sung những đoạn tóm tắt đã cải thiện kết quả ở kỹ thuật

LBC. Không chỉ có kỹ thuật LBC, việc kết hợp RAPTOR với kỹ thuật BSC cũng làm cải thiện chất lượng đáng kể với hai độ đo Rouge-L và F1, cụ thể với độ đo Rouge-L tăng 3.85% và tăng 3.08% với độ đo F1. Thực nghiệm cũng cho thấy, số lượng ngữ cảnh đầu vào trung bình của một câu hỏi giảm đi khi áp dụng RAPTOR vào các kỹ thuật phân đoạn. Cụ thể, đối với kỹ thuật BSC số lượng ngữ cảnh trung bình là 2530 tokens trong khi đó với kỹ thuật BSC kết hợp kỹ thuật RAPTOR chỉ là 2079 tokens. Điều này càng khẳng định việc bổ sung những đoạn văn bản tóm tắt chứa ngữ cảnh toàn cụ giúp cải thiện hiệu năng so với các kỹ thuật thông thường.

4.6 Thử nghiệm và đánh giá của hiệu quả của các phương pháp hợp nhất phân đoạn.

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	Rouge-L	F1	LLM-Score
No Merge (RAPTOR(LBC))	42.6	24.9	16.57	12.8	50.41	49.52	59
Merge 200 tokens	42.88	25.51	17,72	12.85	50.79	50	59
Merge 300 tokens	45.07	26.77	19.03	13.67	53.68	52.99	62.5
Merge 400 tokens (PPDX)	**46.64**	**28.32**	**19.93**	**14.41**	**55.41**	**54.07**	**63.5**
Merge 500 tokens	45,81	27.89	19.52	14.3	53.59	52.94	62

Bảng 4.5: Hiệu năng của các phương pháp hợp nhất phân đoạn

Bảng 4.5 mô tả kết quả khi so sánh hiệu năng của các phương pháp hợp nhất phân đoạn khác nhau. Tất cả các phương pháp đều được thử nghiệm chung một mô hình nhúng, một mô hình xếp hạng và mô hình sinh. Số liệu của từng phương pháp cũng là kết quả cao nhất trong các trường hợp thử nghiệm với top k khác nhau.

Từ bảng trên, kết quả cho thấy việc áp dụng kỹ thuật hợp nhất đã cải thiện đáng kể hiệu năng so với phương pháp không hợp nhất. Cụ thể, phương pháp *Merge 400 token (PPDX)* cho kết quả tốt nhất ở hầu hết các tiêu chí đánh giá, bao gồm Bleu-1, Bleu-2, Bleu-3, Bleu-4, Rouge-L, F1, và LLM-Score. So với phương pháp *No Merge (RAPTOR(LBC))*, việc hợp nhất 400 tokens giúp tăng độ đo Rouge-L lên 9.91% và F1 lên 9.18%, chứng tỏ rằng hợp nhất phân đoạn đã giúp cải thiện đáng kể chất lượng câu trả lời.

Bên cạnh đó, các phương pháp hợp nhất khác như *Merge 200 tokens* và *Merge 300 tokens* cũng ghi nhận sự cải thiện so với *No Merge*, nhưng hiệu quả thấp hơn so với hợp nhất 400 tokens. Tuy nhiên, khi tăng ngưỡng lên 500 tokens, các độ đo

Bleu, có xu hướng giảm nhẹ, độ đo Rouge-L và F1 lần lượt giảm 3.2% và 2.08% trong khi lượng tokens đầu vào trung bình cho một truy vấn của phương pháp này lên tới 2827 tokens cao hơn nhiều so với 2243 tokens của phương pháp đề xuất cho thấy rằng việc hợp nhất quá mức có thể dẫn đến mất cân bằng thông tin, chứa nhiều thông tin dư thừa.

4.7 Thử nghiệm và đánh giá mức độ ảnh hưởng của các chiến lược tìm kiếm và xếp hạng lại (rerank) khác nhau đối với giải pháp đề xuất

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	Rouge-L	F1	LLM-Score
Vector Search + Rerank	41.57	24.24	16.43	12.23	49.68	48.1	55.5
BM25 + Rerank	42.04	26.11	18,15	12.97	51.30	49.44	59.5
HybridSearch + Rerank (PPDX)	**46.64**	**28.32**	**19.93**	**14.41**	**55.41**	**54.07**	**63.5**

Bảng 4.6: Hiệu năng của các chiến lược tìm kiếm

Bảng 4.6 mô tả kết quả khi so sánh hiệu năng của các chiến lược tìm kiếm (retrieve) và xếp hạng lại (rerank) khác nhau.Tất cả các phương pháp đều được thử nghiệm chung một mô hình nhúng, một mô hình xếp hạng và mô hình sinh. Số liệu của từng phương pháp cũng là kết quả cao nhất trong các trường hợp thử nghiệm với top k khác nhau.

Từ bảng trên, kết quả cho thấy rằng phương pháp Hybrid Search + Rerank (PPDX) đã cải thiện đáng kể hiệu năng so với các phương pháp còn lại. Cụ thể, phương pháp này cho thấy sự cải thiện vượt trội ở các độ đo quan trọng như Rouge-L (tăng 11.4% so với phương pháp *Vector Search+Rerank* và 8.1% so với *BM25 search + Rerank*) và F1 (tăng 12.41% so với *Vector Search+Rerank* và 9.36% so với *BM25 search + Rerank*).

Kết chương Chương này đã trình bày chi tiết về thiết kế thử nghiệm và cung cấp những thông tin đánh giá, so sánh về giải pháp đề xuất với các phương pháp tham chiếu. Số liệu từ các độ đo chỉ ra rằng, giải pháp đề xuất đã tạo ra chất lượng câu trả lời tốt hơn so với các kỹ thuật tham chiếu khác. Tuy nhiên, việc sử dụng nhiều mô hình ngôn ngữ lớn và các kỹ thuật chỉ được đánh giá qua chất lượng của câu trả lời được sinh ra từ mô hình ngôn ngữ lớn chưa đem lại được đánh giá chuẩn xác nhất về ảnh hưởng của các kỹ thuật phân đoạn tài liệu khác nhau tác động lên câu trả lời. Vấn đề này, và còn nhiều vấn đề khác mà đồ án chưa giải quyết được, sẽ được nêu rõ và định hướng phát triển trong tương lai ở Chương 5 - chương cuối

cùng của báo cáo đồ án nghiên cứu.

CHƯƠNG 5. KẾT LUẬN

Các chương trước đã trình bày chi tiết toàn bộ quá trình thực hiện và kết quả đạt được trong đồ án nghiên cứu. Chương cuối cùng này sẽ tập trung tổng kết các nội dung đã triển khai, đồng thời phân tích những hạn chế còn tồn tại và các vấn đề chưa được giải quyết triệt để. Những phân tích này sẽ đóng vai trò làm nền tảng để đề xuất các hướng phát triển và cải tiến trong tương lai.

5.1 Kết luận

Nội dung đã đạt được

Dưới sự hướng dẫn tận tình của **PGS. TS. Nguyễn Thị Kim Anh**, đồ án nghiên cứu của tôi đã cơ bản hoàn thành các mục tiêu đề ra. Cụ thể, nghiên cứu đã tiến hành phân tích về mô hình ngôn ngữ lớn (Large Language Models - LLMs), tập trung vào việc sử dụng LLM để sinh câu trả lời trong các hệ thống hỏi đáp. Đồ án cũng thảo luận các ứng dụng thực tiễn, đồng thời đánh giá những thách thức và hạn chế của LLM trong việc đảm bảo chất lượng đầu ra. Đặc biệt, nghiên cứu đã làm rõ vai trò của kỹ thuật RAG (Retrieval-Augmented Generation) trong việc khắc phục những hạn chế này, đồng thời phân tích sâu từng mô-đun của kỹ thuật để hiểu rõ cơ chế hoạt động. Ngoài ra, tôi đã tìm hiểu, tham khảo và tổng hợp các công trình liên quan, từ đó đề xuất giải pháp cải tiến nhằm nâng cao chất lượng đầu ra của hệ thống. Việc thực hiện đồ án nghiên cứu, ngoài việc được trau dồi thêm các kiến thức mới, tôi còn rèn luyện được các kỹ năng phân tích, tổng hợp thông tin và áp dụng các giải pháp nghiên cứu khoa học vào thực tiễn. Quá trình này giúp tôi hiểu rõ hơn về cách thức hoạt động của các mô hình ngôn ngữ lớn cũng như những ứng dụng tiềm năng của chúng trong nhiều lĩnh vực. Hơn nữa, thông qua việc đánh giá các thách thức và hạn chế của LLM, tôi đã được tiếp xúc với nhiều kỹ thuật mới, những công nghệ là xu hướng của tương lai.

Hạn chế của giải pháp đề xuất

Mặc dù giải pháp được đề xuất đã cho thấy sự cải tiến đáng kể, tuy nhiên vẫn còn tồn tại nhiều hạn chế. Do thời gian thực hiện đồ án bị giới hạn cùng với kiến thức hiện tại của tôi chưa đầy đủ, việc áp dụng các kỹ thuật mới đã khiến tôi chưa thể giải quyết triệt để các vấn đề này.

Như tôi đã đề cập ở phần 4.6, việc sử dụng các mô hình ngôn ngữ lớn để phân đoạn khiến tôi khó đánh giá được chất lượng của những phân đoạn. Thêm vào đó, những mô hình được sử dụng là những mô hình với lượng tham số nhỏ như: Gwen-1.5B-Instruct hoặc là mô hình Gemini cung cấp API hỗ trợ, chúng thường không

ổn định hoặc là bị giới hạn số lượng yêu cầu mỗi ngày. Sự thiếu ổn định này cũng khiến quá trình thử nghiệm và tối ưu hóa mô hình gặp nhiều trở ngại, hạn chế khả năng cải tiến liên tục của hệ thống.

Bên cạnh đó, sự liên quan và thích đáng của những phân đoạn sau khi truy xuất chưa có sự đánh giá đầy đủ và chi tiết. Đôi khi, việc truy xuất ra thừa quá nhiều đoạn văn bản không hỗ trợ cho câu trả lời khiến mô hình sinh tạo ra những câu trả lời sai.

5.2 Hướng phát triển trong tương lai

Với những hạn chế đã trình bày ở Phần 5.1, trong tương lai, ta có thể phát triển hoặc sử dụng thêm một số kỹ thuật khác để giải quyết các vấn đề còn tồn đọng.

Đầu tiên, trong hầu hết các kỹ thuật phân đoạn thông thường đều bị mất thông tin ngữ cảnh do sử dụng nhiều các đại từ thay thế trong tài liệu minh hoạ ở hình 1.2. Để đảm bảo duy trì được thông tin đối tượng, tôi đề xuất sử dụng phân giải đồng tham chiếu để khôi phục các đại từ thay thế về tên cụ thể của đối tượng.

Tiếp đó, việc sử dụng mô hình ngôn ngữ lớn cho phân đoạn cũng như tạo văn bản tóm tắt phụ thuộc vào năng lực của mô hình. Do hạn chế về tài nguyên tính toán mà kết quả phân đoạn cũng như bản tóm tắt còn chưa chính xác, ảnh hưởng lớn đến kết quả đánh giá của phương pháp đề xuất. Do đó, trong tương lai có thể sử dụng các mô hình có năng lực trong việc phân đoạn và tóm tắt tốt hơn. Trong một số trường hợp yêu cầu độ chính xác cao, có thể cho phép sự tham gia của con người vào quá trình tóm tắt.

Bên cạnh đó, do thời gian có hạn, đồ án chỉ thực hiện thử nghiệm trên tập dữ liệu nhỏ nên có thể chưa đánh giá được đầy đủ hiệu suất của giải pháp đề xuất. Hơn nữa, đồ án chưa đánh giá được chất lượng phân đoạn sau khi truy xuất do thiếu bộ dữ liệu đánh giá về tính chính xác của phân đoạn truy xuất được. Trong tương lai, đề xuất tìm kiếm và xây dựng thêm nhiều bộ dữ liệu khác nhau để có góc nhìn tổng quan về giải pháp đề xuất.

TÀI LIỆU THAM KHẢO

[1] A. Grattafiori, A. Dubey, A. Jauhri **andothers**, *The llama 3 herd of models*, 2024. arXiv: 2407.21783 [cs.AI]. **url**: https://arxiv.org/abs/2407.21783.

[2] S. Minaee, T. Mikolov, N. Nikzad **andothers**, *Large language models: A survey*, 2024. arXiv: 2402.06196 [cs.CL]. **url**: https://arxiv.org/abs/2402.06196.

[3] L. Huang, W. Yu, W. Ma **andothers**, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, **november** 2024, ISSN: 1558-2868. DOI: 10.1145/3703155. **url**: http://dx.doi.org/10.1145/3703155.

[4] OpenAI, : A. Hurst **andothers**, *Gpt-4o system card*, 2024. arXiv: 2410.21276 [cs.CL]. **url**: https://arxiv.org/abs/2410.21276.

[5] Z. Xu, S. Jain **and** M. Kankanhalli, *Hallucination is inevitable: An innate limitation of large language models*, 2024. arXiv: 2401.11817 [cs.CL]. **url**: https://arxiv.org/abs/2401.11817.

[6] K. Shuster, S. Poff, M. Chen, D. Kiela **and** J. Weston, *Retrieval augmentation reduces hallucination in conversation*, 2021. arXiv: 2104.07567 [cs.CL]. **url**: https://arxiv.org/abs/2104.07567.

[7] P. Finardi, L. Avila, R. Castaldoni **andothers**, *The chronicles of rag: The retriever, the chunk and the generator*, 2024. arXiv: 2401.07883 [cs.LG]. **url**: https://arxiv.org/abs/2401.07883.

[8] LlamaIndex, *Semantic chunker*, 2024. **url**: https://docs.llamaindex.ai/en/stable/examples/node_parsers/semantic_chunking/.

[9] I. S. Singh, R. Aggarwal, I. Allahverdiyev **andothers**, *Chunkrag: Novel llm-chunk filtering method for rag systems*, 2024. arXiv: 2410.19572 [cs.CL]. **url**: https://arxiv.org/abs/2410.19572.

[10] R. Qu, R. Tu **and** F. Bao, *Is semantic chunking worth the computational cost?* 2024. arXiv: 2410.13070 [cs.CL]. **url**: https://arxiv.org/abs/2410.13070.

[11] A. Vaswani, N. Shazeer, N. Parmar **andothers**, *Attention is all you need*, 2023. arXiv: 1706.03762 [cs.CL]. **url**: https://arxiv.org/abs/1706.03762.

[12] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie **and** C. D. Manning, *Raptor: Recursive abstractive processing for tree-organized retrieval*, 2024.

arXiv: 2401.18059 [cs.CL]. **url**: https://arxiv.org/abs/2401.18059.

[13] Y. Gao, Y. Xiong, X. Gao **andothers**, *Retrieval-augmented generation for large language models: A survey*, 2024. arXiv: 2312.10997 [cs.CL]. **url**: https://arxiv.org/abs/2312.10997.

[14] Qwen, : A. Yang **andothers**, *Qwen2.5 technical report*, 2024. arXiv: 2412.15115 [cs.CL]. **url**: https://arxiv.org/abs/2412.15115.

[15] G. Team, P. Georgiev, V. I. Lei **andothers**, *Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context*, 2024. arXiv: 2403.05530 [cs.CL]. **url**: https://arxiv.org/abs/2403.05530.

[16] A. V. Duarte, J. Marques, M. Graça, M. Freire, L. Li **and** A. L. Oliveira, *Lumberchunker: Long-form narrative document segmentation*, 2024. arXiv: 2406.17526 [cs.CL]. **url**: https://arxiv.org/abs/2406.17526.

[17] X. H. Lù, *Bm25s: Orders of magnitude faster lexical search via eager sparse scoring*, 2024. arXiv: 2407.03618 [cs.IR]. **url**: https://arxiv.org/abs/2407.03618.

[18] K. Papineni, S. Roukos, T. Ward **and** W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation," **in***Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* P. Isabelle, E. Charniak **and** D. Lin, **editors**, Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, **july** 2002, **pages** 311–318. DOI: 10.3115/1073083.1073135. **url**: https://aclanthology.org/P02-1040/.

[19] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," **in***Text Summarization Branches Out* Barcelona, Spain: Association for Computational Linguistics, **july** 2004, **pages** 74–81. **url**: https://aclanthology.org/W04-1013/.

[20] Y. Tang **and** Y. Yang, *Multihop-rag: Benchmarking retrieval-augmented generation for multi-hop queries*, 2024. arXiv: 2401.15391 [cs.CL]. **url**: https://arxiv.org/abs/2401.15391.

[21] L. Zheng, W.-L. Chiang, Y. Sheng **andothers**, *Judging llm-as-a-judge with mt-bench and chatbot arena*, 2023. arXiv: 2306.05685 [cs.CL]. **url**: https://arxiv.org/abs/2306.05685.

[22] Y. Bai, X. Lv, J. Zhang **andothers**, *Longbench: A bilingual, multitask benchmark for long context understanding*, 2024. arXiv: 2308.14508 [cs.CL]. **url**: https://arxiv.org/abs/2308.14508.

[23] Q. Zhao, R. Wang, Y. Cen **andothers**, *Longrag: A dual-perspective retrieval-augmented generation paradigm for long-context question answering*, 2024. arXiv: 2410.18050 [cs.CL]. **url**: https://arxiv.org/abs/2410.18050.

PHỤ LỤC

A. Hiệu quả tính toán của giải pháp đề xuất và các kỹ thuật tham chiếu

	Top k	Avg time/Question	Avg Context/Question	F1
PPDX	top 7	3.35	2243	54.07
BSC	top 5	4	2530	51.53
RAPTOR	top 10	2.63	1843	50.5
FSC	top 10	4.4	2817	49.31

Bảng A.1: Thời gian sinh câu trả lời và lượng ngữ cảnh đầu vào

Bảng A.1 mô tả thời gian trung bình để sinh câu trả lời của các kỹ thuật thử nghiệm và giải pháp đề xuất khi đạt kết quả cao nhất ở các top k khác nhau trong các lần thử nghiệm. Từ đó thấy được mặc dù thời gian sinh câu trả lời của giải pháp đề xuất cao hơn kỹ thuật RAPTOR nhưng lại cải thiện kết quả đáng kể. So với kỹ thuật FSC và BSC, giải pháp đề xuất vừa có thời gian sinh câu trả lời ngắn hơn vừa có hiệu năng cao hơn.

	Thời gian trung bình phân đoạn một tập tài liệu
PPDX	12.57
BSC	3.24
RAPTOR	6.24
FSC	0.34

Bảng A.2: Thời gian phân đoạn trung bình môt tài liệu trong tập dữ liệu thử nghiệm

Bảng A.2 mô tả thời gian trung bình để phân đoạn một tài liệu trong tập dữ liệu thử nghiêm. Do sử dụng mô hình ngôn ngữ lớn nên thời gian phân đoạn của giải pháp đề xuất cao hơn đáng kể so với các kỹ thuật BSC và FSC.

B. Một số ví dụ thực nghiệm

Hinh_ve/AƠẨnh maƠÀn hiƠÀnh 2025-01-05 luƠÁc 11.08.31.png

Hình B.1: Ví dụ khi thử nghiệm

Hình B.1 mô tả một ví dụ của câu trả lời khi sử dụng về kỹ thuật *Breakpoint-semantic Chunking* với phần màu xanh là thông tin liên quan sử dụng để trả lời câu hỏi và màu đỏ là ngược lại

Hinh_ve/AƠẨnh maƠÀn hiƠÀnh 2025-01-05 luƠÁc 11.36.09.png

Hình B.2: Ví dụ khi thử nghiệm với câu trả lời của giải pháp đề xuất

Hình B.2 mô tả một ví dụ của câu trả lời khi sử dụng về giải pháp đề xuất với phần màu xanh là thông tin liên quan sử dụng để trả lời câu hỏi và màu đỏ là ngược lại